Computer Organization and Operating System

UAMPC0502

Credit: 2

ISE1	**10**
MSE	**30**
ISE2	**10**
ESE	**50**

Overview

1. Introduction to Computer Organization
2. Introduction to Operating Systems- IPC,Synchronization
3. Deadlocks & File Management
4. Memory Management

	Story	Real-Life Scenario	OS Concepts	Computer Organization Concepts	Why It's Interesting
1	**Ever wonder how Netflix can stream to millions of users without crashing or lagging?**	Netflix streams to millions with no lag	Process scheduling, thread management, concurrency	Multi-core CPUs, cache hierarchy, instruction execution	Shows how OS + CPU design make real-time video possible
2	**When you say "Hey Siri," your phone wakes up from a low-power state, loads the voice recognition engine, and responds — all within milliseconds.**	"Hey Siri" triggers real-time voice recognition	Interrupts, process states, I/O management	ALUs, control units, low-power CPU states	Demonstrates efficiency of hardware-software collaboration
3	**Tesla Model 3 processes data from dozens of sensors in real-time. It needs to decide within milliseconds whether to brake, accelerate, or turn.**	Robots in warehouses make constant decisions	Process/thread scheduling, IPC, synchronization	DMA, CPU time-sharing, system buses	Business-critical — faster systems = higher profits
4	**Smartphones Stay Cool**	Phones manage performance without overheating	Task scheduling, power-aware OS features	ARM architecture, pipeline efficiency, instruction sets	How smartphones balance power, speed, and temperature

Introduction to Computer Organization

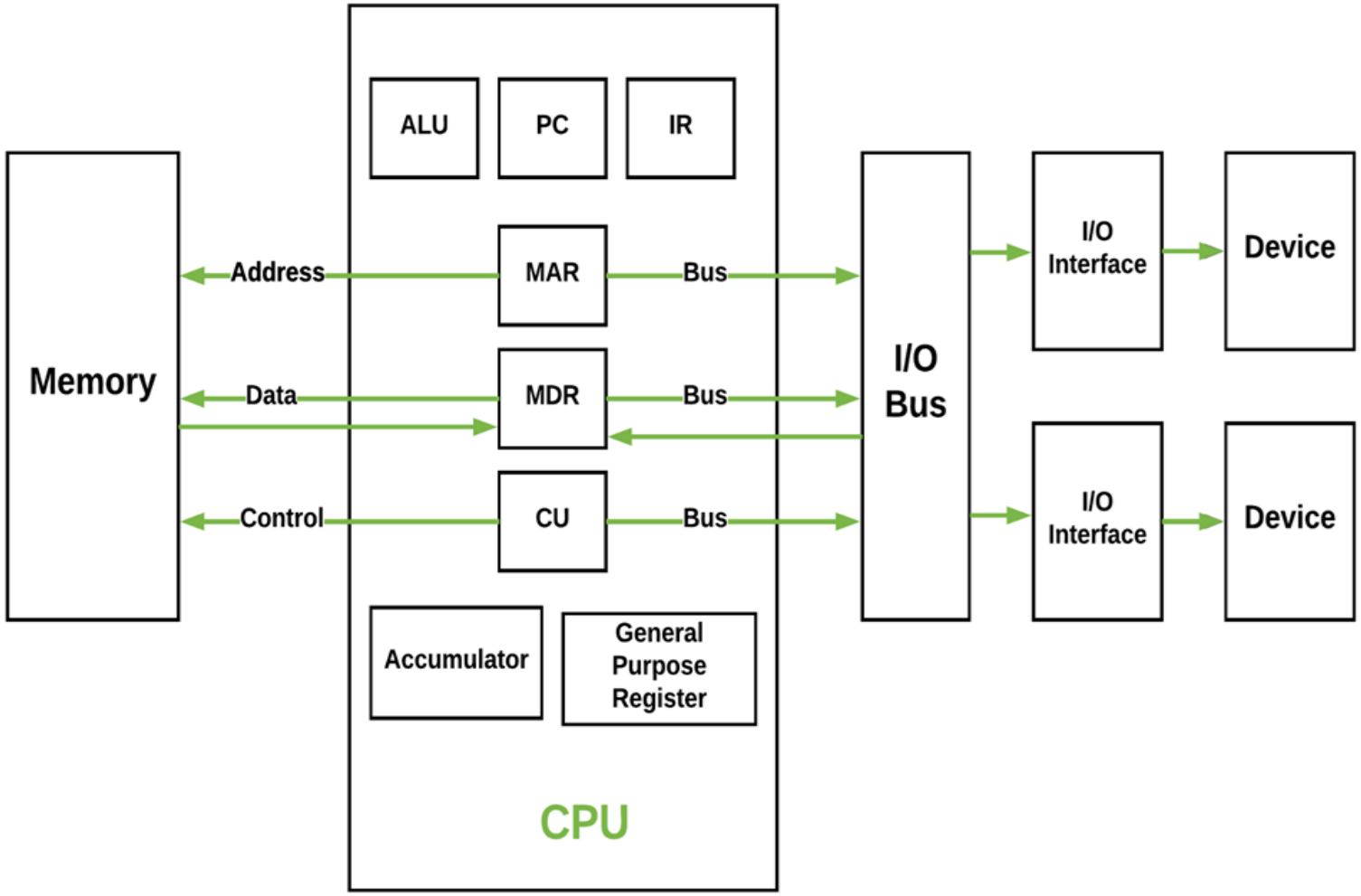
Outline

- Introduction to RISC and CISC architectures,
- ALU and control unit,
- Hardwired vs. Microprogrammed control,
- Pipelining and its performance,
- Memory organization and types,
- Cache memory: Mapping techniques and replacement policies

Why Computer Organization?

- Focuses on the internal structure and functional behavior of a computer system.
- It explains how different components (CPU, memory, I/O, buses) work together to execute instructions.
- Design better programs, including system software such as compilers, operating systems, and device drivers.
- Optimize program behaviour.
- Evaluate (benchmark) computer system performance.
- Understand time, space, and price tradeoffs.

Basic Computer Organization



- Control Unit
- Arithmetic and Logic Unit (ALU)
- Main Memory Unit (Registers) -

  a. Accumulator

  b. Program Counter (PC)

  c. Memory Address Register (MAR)

  d. Memory Data Register (MDR)

  e. Current Instruction Register (CIR)

  f. Instruction Buffer Register (IBR)

- Input/Output Devices
- Buses

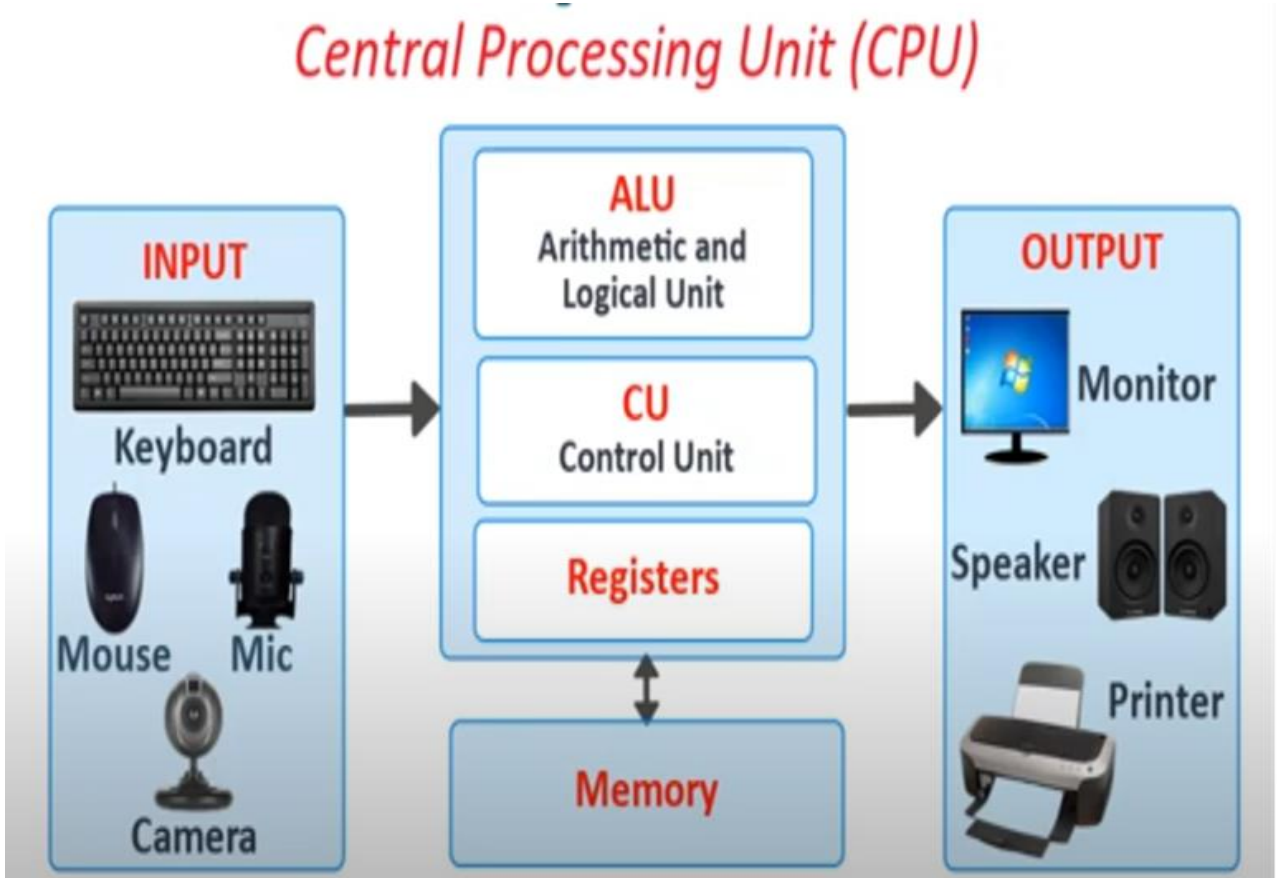
  - Data Bus

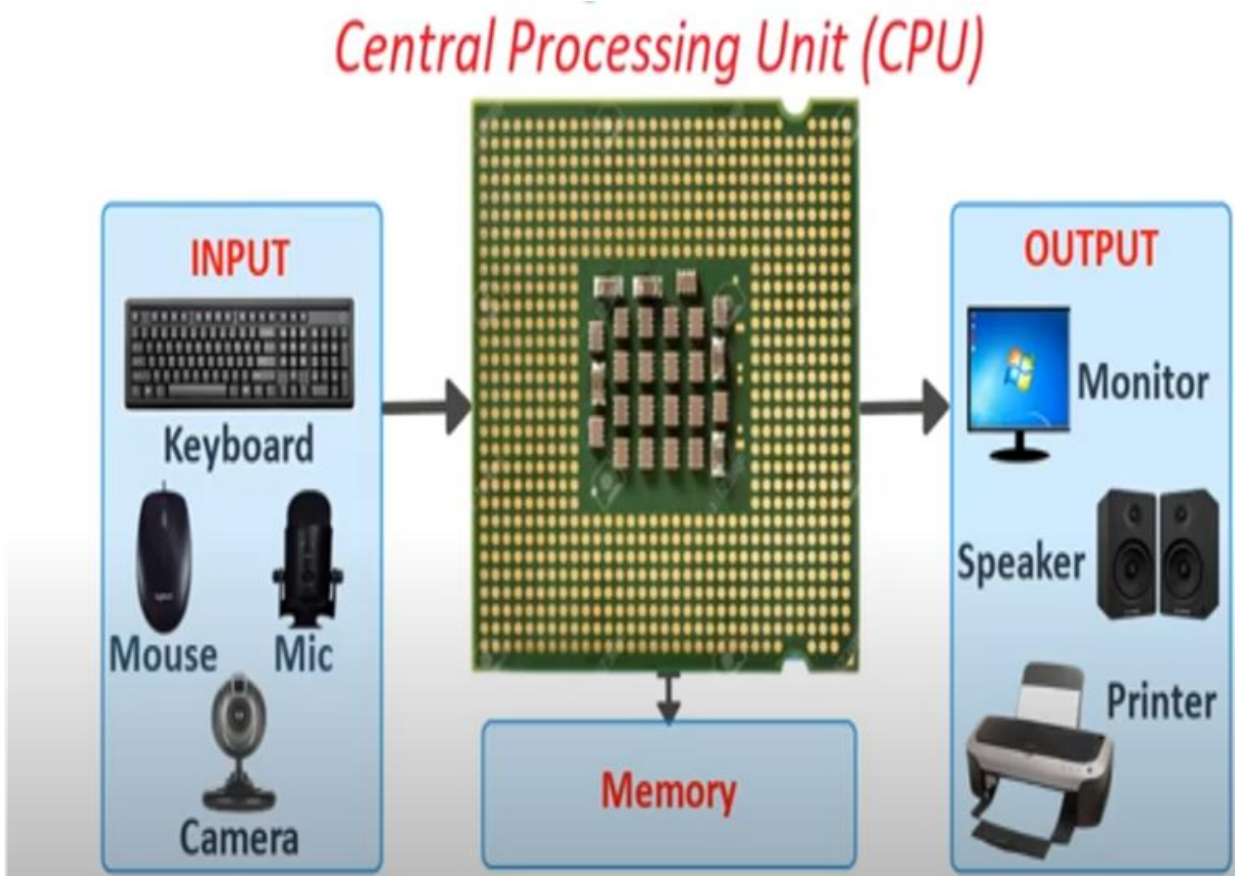
  - Address Bus

  - Control Bus

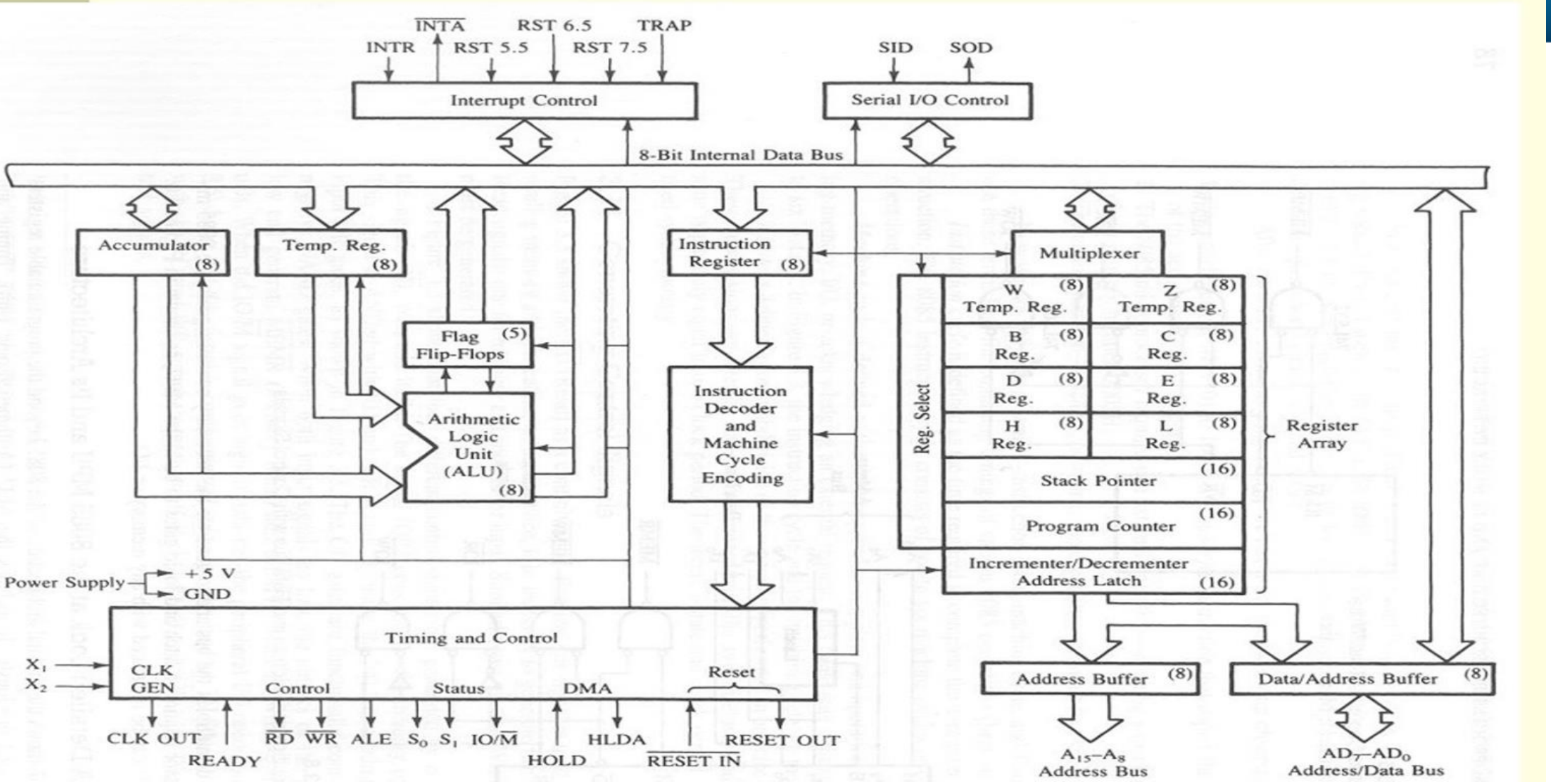
Microprocessor

A microprocessor is an integrated circuit that incorporates the central processing unit of a computer on a single chip. It takes binary input from devices, processes it according to instructions stored in memory, and provides output.





Architecture of 8085

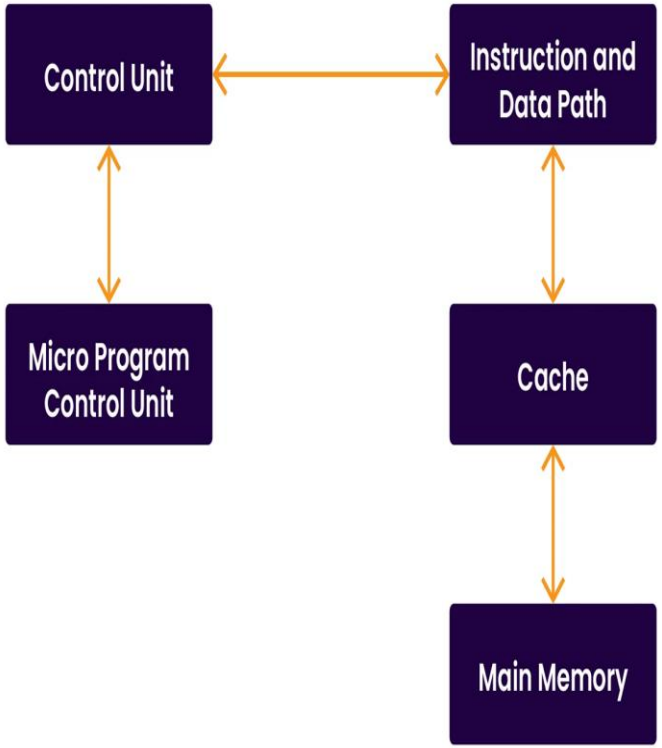


Working with example

RISC vs. CISC Architectures

- **RISC (Reduced Instruction Set Computer):**
  - Simple, uniform instructions
  - Fewer addressing modes
  - High performance via pipelining
  - Example: ARM, MIPS
- **CISC (Complex Instruction Set Computer):**
  - Rich set of complex instructions
  - More addressing modes
  - Emphasis on hardware simplicity for compiler
  - Example: Intel x86

Introduction: CISC



- CISC means Complex Instruction Set Computer chips that are easy to program and which make efficient use of memory. Since the earliest machines were programmed in assembly language and memory was slow and expensive, the CISC philosophy was commonly implemented in large computers as the PDP-11 and the DEC system 10 and 20 machines.
- Most common microprocessor designs such as the Intel 80x86 and Motorola 68K series followed the CISC philosophy.
- CISC was developed to make compiler development simpler. It shifts most of the burden of generating machine instructions to the processor. For example, instead of having to make a compiler write long machine instructions to calculate a square-root, a CISC processor would have a built-in ability to do this.

CISC Attributes

- CISC instructions sets have some common characteristics:
- A 2-operand format, where instructions have a source and a destination. Register to register, register to memory, and memory to register commands.
- Variable length instructions where the length often varies according to the addressing mode.
- Instructions which require multiple clock cycles to execute.
- E.g. Pentium is considered a modern CISC processor

- Most CISC hardware architectures have several characteristics in common:
- Complex instruction-decoding logic, driven by the need for a single instruction to support multiple addressing modes.
- A small number of general purpose registers. This is the direct result of having instructions which can operate directly on memory and the limited amount of chip space not dedicated to instruction decoding, execution, and microcode storage.
- Several special purpose registers. Many CISC designs set special registers for the stack pointer, interrupt handling, and so on.
- A “Condition code” register which is set as a side-effect of most instructions. This register reflects whether the result of the last operation is less than, equal to, or greater than zero and records if certain error conditions occur.

At the time of their initial development, CISC machines used available technologies to optimize computer performance.

- Microprogramming is as easy as assembly language to implement, and much less expensive than hardwiring a control unit.
- The ease of microcoding new instructions allowed designers to make CISC machines upwardly compatible: a new computer could run the same programs as earlier computers because the new computer would contain a superset of the instructions of the earlier computers.
- As each instruction became more capable, fewer instructions could be used to implement a given task. This made more efficient use of the relatively slow main memory.
- Because microprogram instruction sets can be written to match the constructs of high-level languages, the compiler does not have to be as complicated.

Complex Instruction Set Computer (CISC) Characteristics

Major characteristics of a CISC architecture

- A large number of instructions - typically from 100 to 250 instruction
- Some instructions that perform specialized tasks and are used infrequently
- A large variety of addressing modes - typically from 5 to 20

different modes

- Variable-length instruction formats
- Instructions that manipulate operands in memory (RISC in register)

What is RISC?

- RISC, or Reduced Instruction Set Computer. is a type of microprocessor architecture that utilizes a small, highly-optimized set of instructions, rather than a more specialized set of instructions often found in other types of architectures.

- History:

The first RISC projects came from IBM, Stanford, and UC-Berkeley in the late 70s and early 80s. The IBM 801, Stanford MIPS, and Berkeley RISC 1 and 2 were all designed with a similar philosophy which has become known as RISC. Certain design features have been characteristic of most RISC processors:

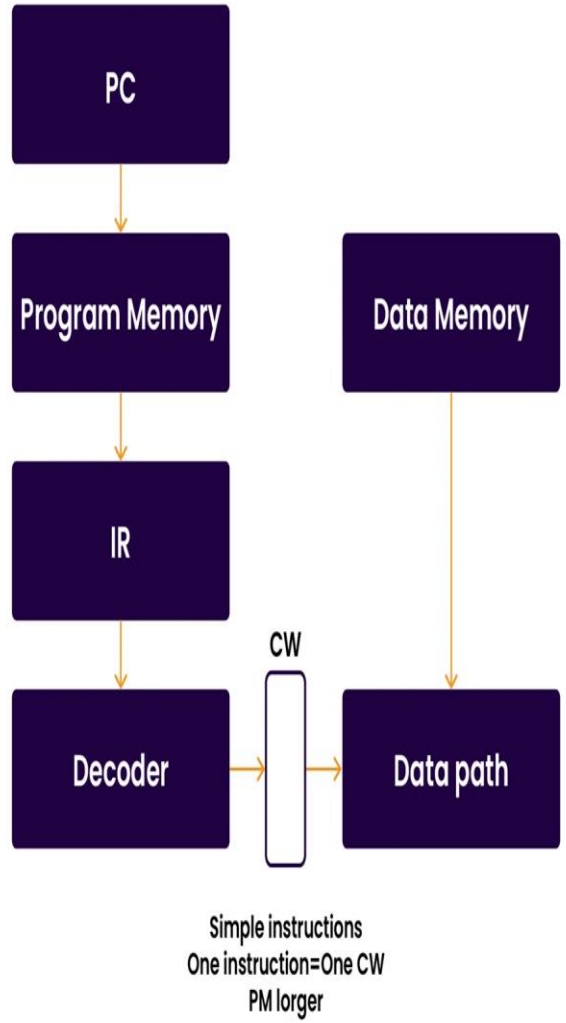
– one cycle execution time: RISC processors have a CPI (clock per instruction) of one cycle. This is due to the optimization of each instruction on the CPU and a technique called PIPELINING.

– pipelining: a techique that allows for simultaneous execution of parts, or stages, of instructions to more efficiently process instructions;

–large number of registers: the RISC design philosophy generally incorporates a larger number of registers to prevent in large amounts of interactions with memory

Reduced Instruction Set Computer (RISC)

- Major characteristics of a RISC architecture
  1. Relatively few instructions
  2. Relatively few addressing modes
  3. Memory access limited to load and store instruction
  4. All operations done within the registers of the CPU
  5. Fixed-length, easily decoded instruction format
  6. Single-cycle instruction execution
  7. Hardwired rather than microprogrammed control



- RISC Instruction
  - Only use LOAD and STORE instruction when communicating between memory and CPU
  - All other instructions are executed within the registers of the CPU without referring to memory

Program to evaluate $X = (A + B) * (C + D)$

LOAD	R1, A	$R1 \leftarrow M[A]$
LOAD	R2, B	$R2 \leftarrow M[B]$
LOAD	R3, C	$R3 \leftarrow M[C]$
LOAD	R4, D	$R4 \leftarrow M[D]$
ADD	R1, R1, R2	$R1 \leftarrow R1 + R2$
ADD	R3, R3, R4	$R3 \leftarrow R3 + R4$
MUL	R1, R1, R3	$R1 \leftarrow R1 * R3$
STORE	X, R1	$M[X] \leftarrow R1$

•Load instruction transfers the operand from memory to CPU Register.

•Add and Multiply operations are executed with data in the registers without accessing the memory.

•Result is then stored in the memory with store information.

- Other characteristics of a RISC architecture:
  1. A relatively large number of registers in the processor unit
  2. Use of overlapped register windows to speed-up procedure call and return
  3. Efficient instruction pipeline
  4. Compiler support for efficient translation of high-level language programs into machine language programs

CISC vs RISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Uses both hardwired and microprogrammed control unit	Uses only Hardwired control unit
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
Small code sizes, high cycles per second	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

CISC vs RISC

CISC	RISC
Variable sized instructions	Fixed sized instructions
Can perform REG to REG or REG to MEM or MEM to MEM	Can perform only Register to Register Arithmetic operations
Requires less number of registers	Requires more number of registers
CISC is Complex Instruction Cycle.	RISC is Reduced Instruction Cycle.
CISC is less pipelined.	RISC is highly pipelined.
Addressing modes are more.	Addressing modes are less.

ALU and Control Unit

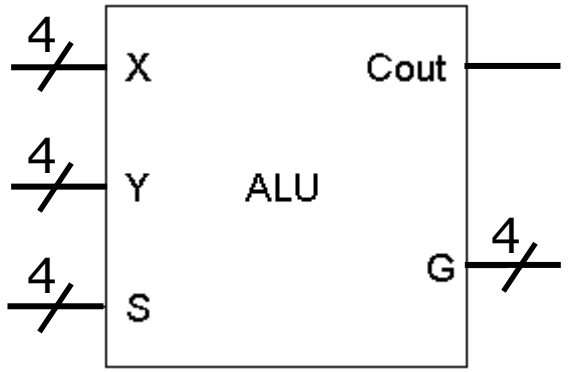
- **ALU (Arithmetic Logic Unit):**
  - Performs arithmetic (add, subtract) and logical (AND, OR) operations
- **Control Unit:**
  - Directs operations of the processor
  - Interprets instructions and signals components

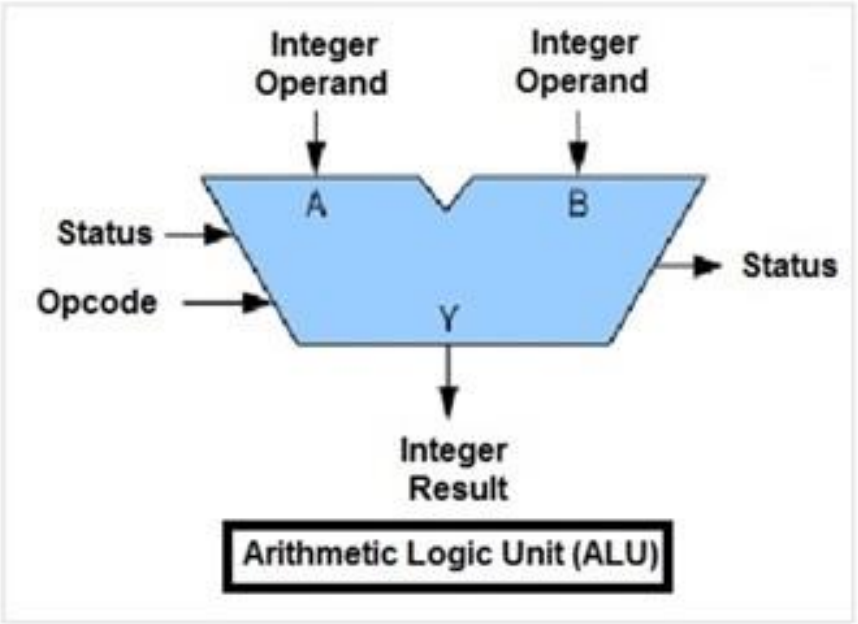
Arithmetic-logic unit

- An arithmetic-logic unit, or ALU, performs many different arithmetic and logic operations. The ALU is the “heart” of a processor—you could say that everything else in the CPU is there to support the ALU.
- Here’s the plan:
  - An arithmetic unit is built off ideas from the adder-subtractor circuit.
  - Then logic operations used to build a logic unit.
  - Finally, putting all these pieces together using multiplexers.

The completed ALU

- This ALU is a good example of hierarchical design.
  - With the 12 inputs, the truth table would have had $2^{12} = 4096$ lines. That's an awful lot of paper.
  - Instead, we were able to use components that we've seen before to construct the entire circuit from a couple of easy-to-understand components.
- As always, we encapsulate the complete circuit in a "black box" so we can reuse it in fancier circuits.

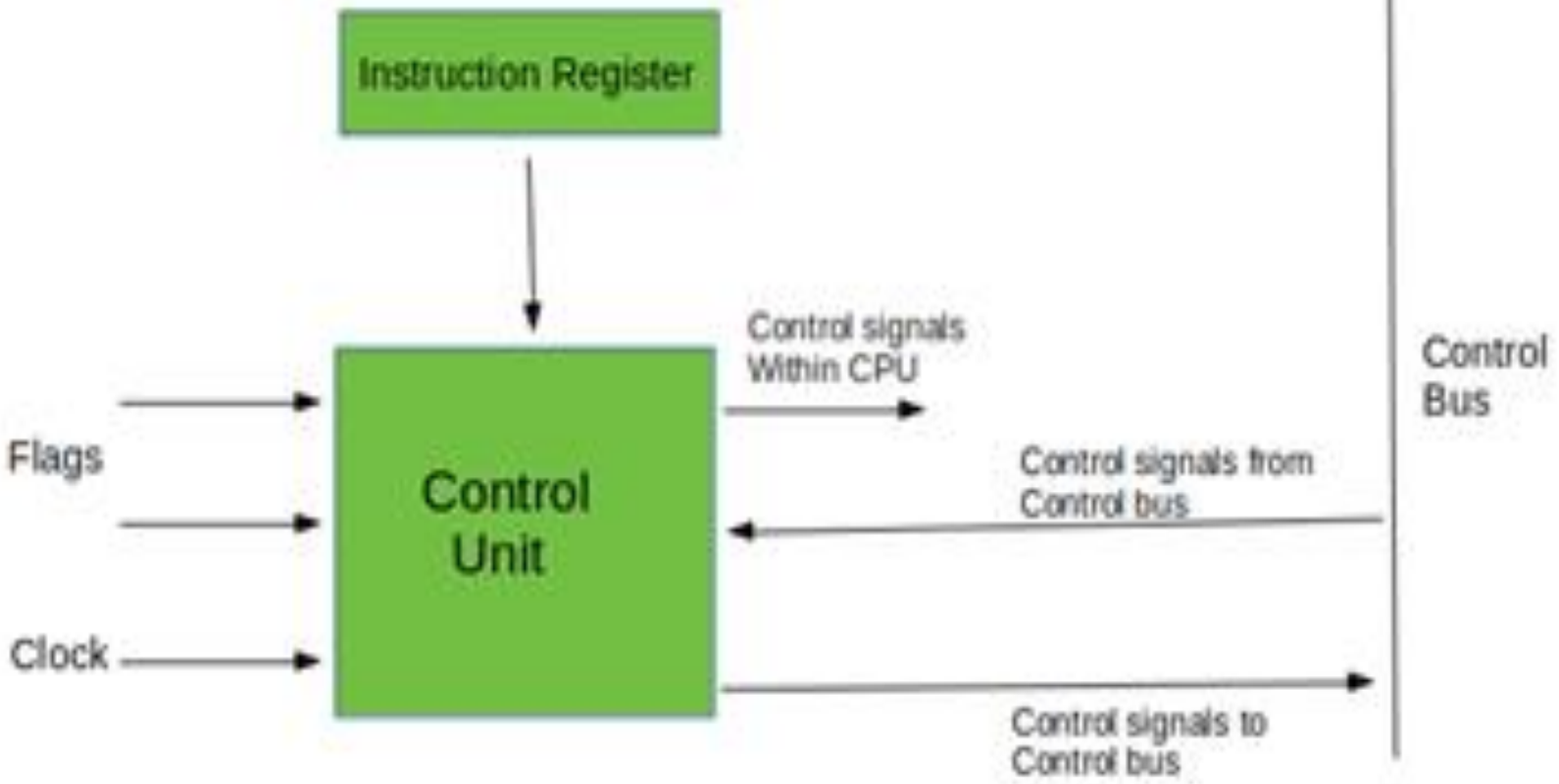




Control unit

Control Unit Functions :

- The control unit (CU) is a component of a CPU that directs the operation of the processor.
- It tells the computer's memory, arithmetic/logic unit(ALU) and input - output devices (I/O) How to respond to a program's instructions ?
- It directs the operation of the other units by providing timing and control signals.
- All computer resources are managed by the CU (Control Unit).
- It directs the flow of data between the CPU and the other devices.
- The control unit communicates / interact with ALU and main memory.
- Control unit generates the control signals to perform the operations.
- It controls the transmission between processor, memory and the various peripherals.
- It also instructs the ALU , which operation should be performed on data.



Block Diagram of the Control Unit

Design of Control Unit

- Control unit can be designed by two methods :

1. **Hardwired Control Unit**
2. **Micro-Programmed Control Unit.**

Hardwired Control Unit:

- It is implemented with the help of Physical components which is Hardwired (gates, flip flops, decoders) in the hardware.
- The inputs to control unit are the instruction register, flags, timing signals.
- If design is modified or changed, all the combinational circuits should be modified which is a very difficult task.
- The sequence of the operation carried out by this machine is determined by wiring of the logic elements so, it is known as “Hardwired”.

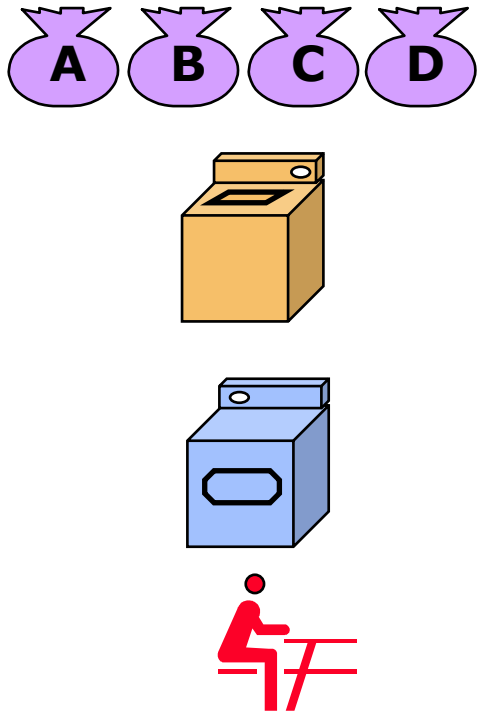
Micro-programmed Control Unit:

- It is implemented by using Programming approach.
- A sequence of micro operations is carried out by executing a program which consist of micro-instructions.
- In this, any modifications or changes can be done by updating the micro program in the control memory which is done by programmer.
- In this ,Control signals are generated by a program.
- Its speed is slow because of the time it takes to fetch microinstructions from control memory.

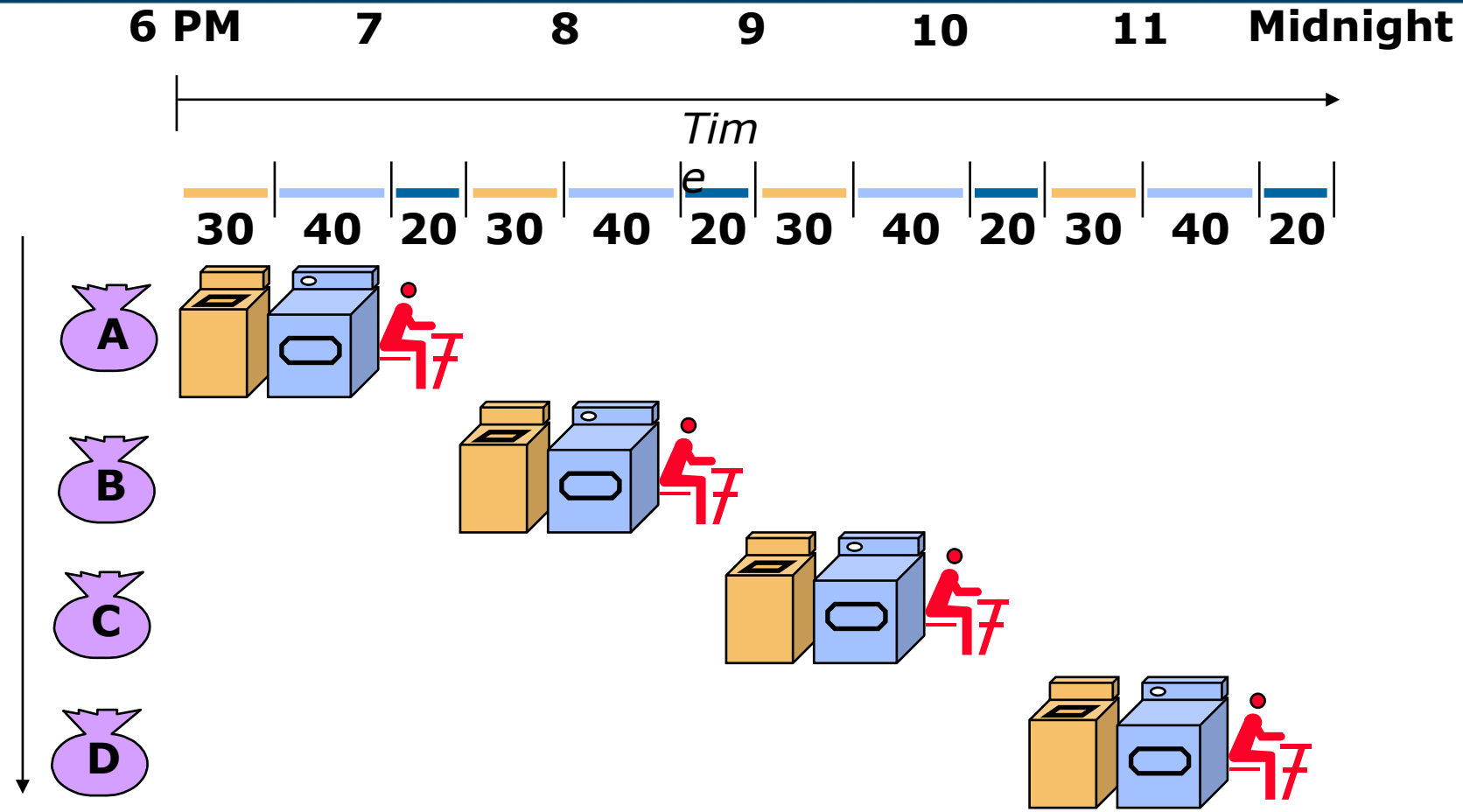
Basis	Hardwired Control Unit	Microprogrammed Control Unit
Basic	It is a circuitry approach.	This control unit is implemented by programming
Design	RISC style instructions	CISC style instructions
Modification	Modification is difficult as the control unit is hardwired. Modifying it will require the change in hardware.	Modifications are easy in case of microprogrammed control unit as it will require the in change in the code only.
Instructions	It works well for simple instructions.	It works well for complex instructions also.
Costing	Implementing hardwired structure requires a cost.	Implementing microprograms is not costly.
Control memory	No control memory is required	Control memory is required
Execution Speed	Faster execution	Comparatively slow

What Is Pipelining?

- Laundry Example
- Ann, Brian, Cathy, Dave each have one load of clothes to wash, dry, and fold
- Washer takes 30 minutes
- Dryer takes 40 minutes
- "Folder" takes 20 minutes



What Is Pipelining?

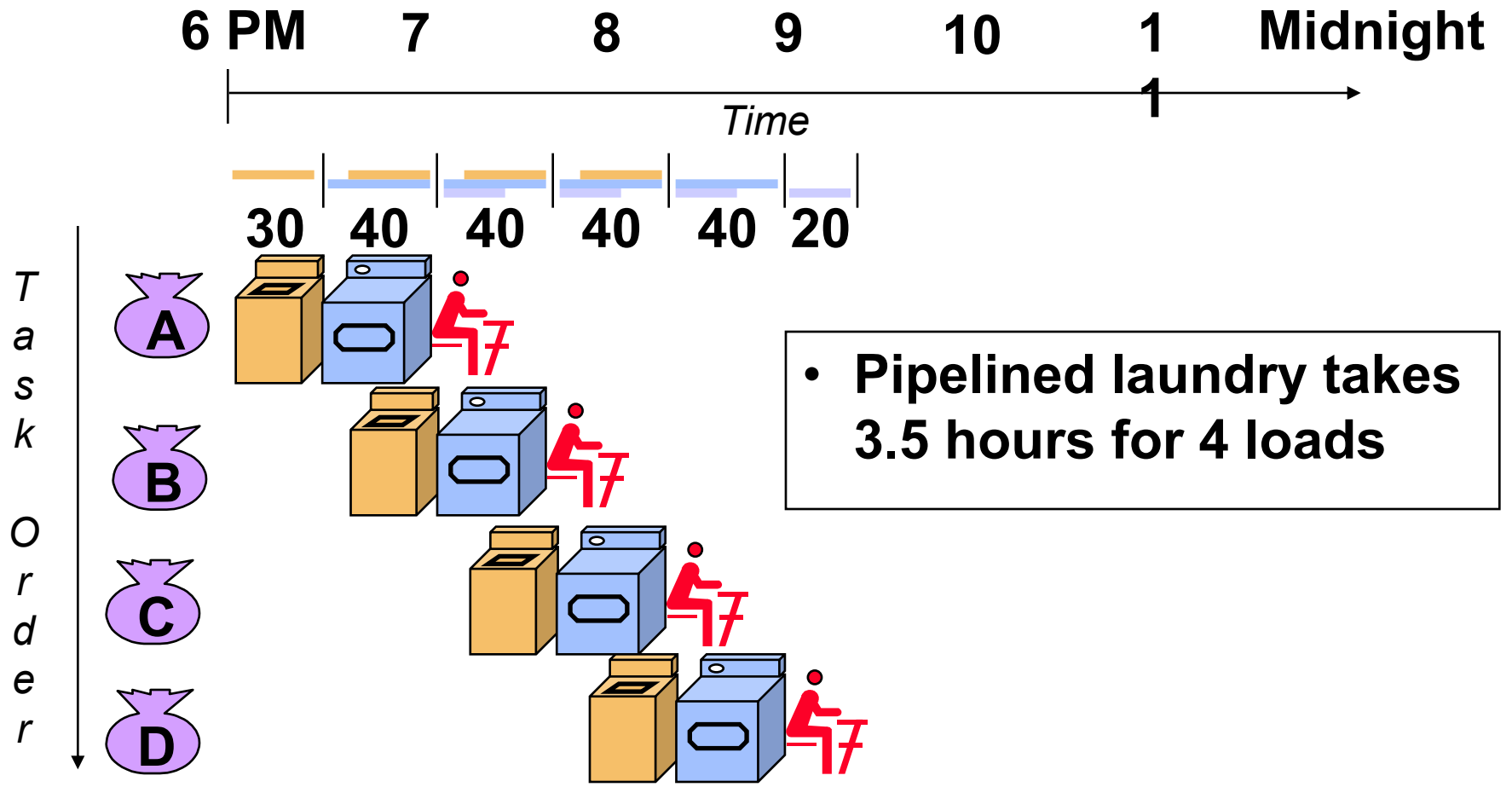


Sequential laundry takes 6 hours for 4 loads

If they learned pipelining, how long would laundry take?

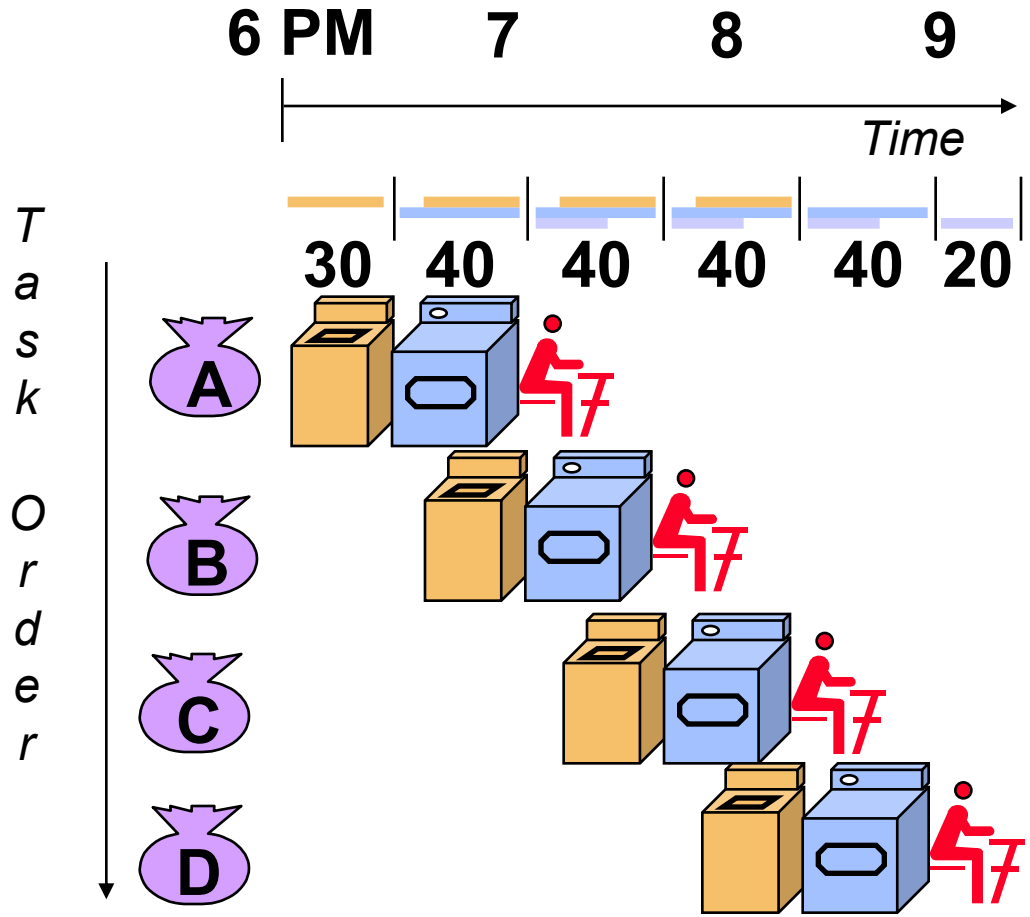
What Is Pipelining

Start work ASAP



- Pipelined laundry takes 3.5 hours for 4 loads

What Is Pipelining

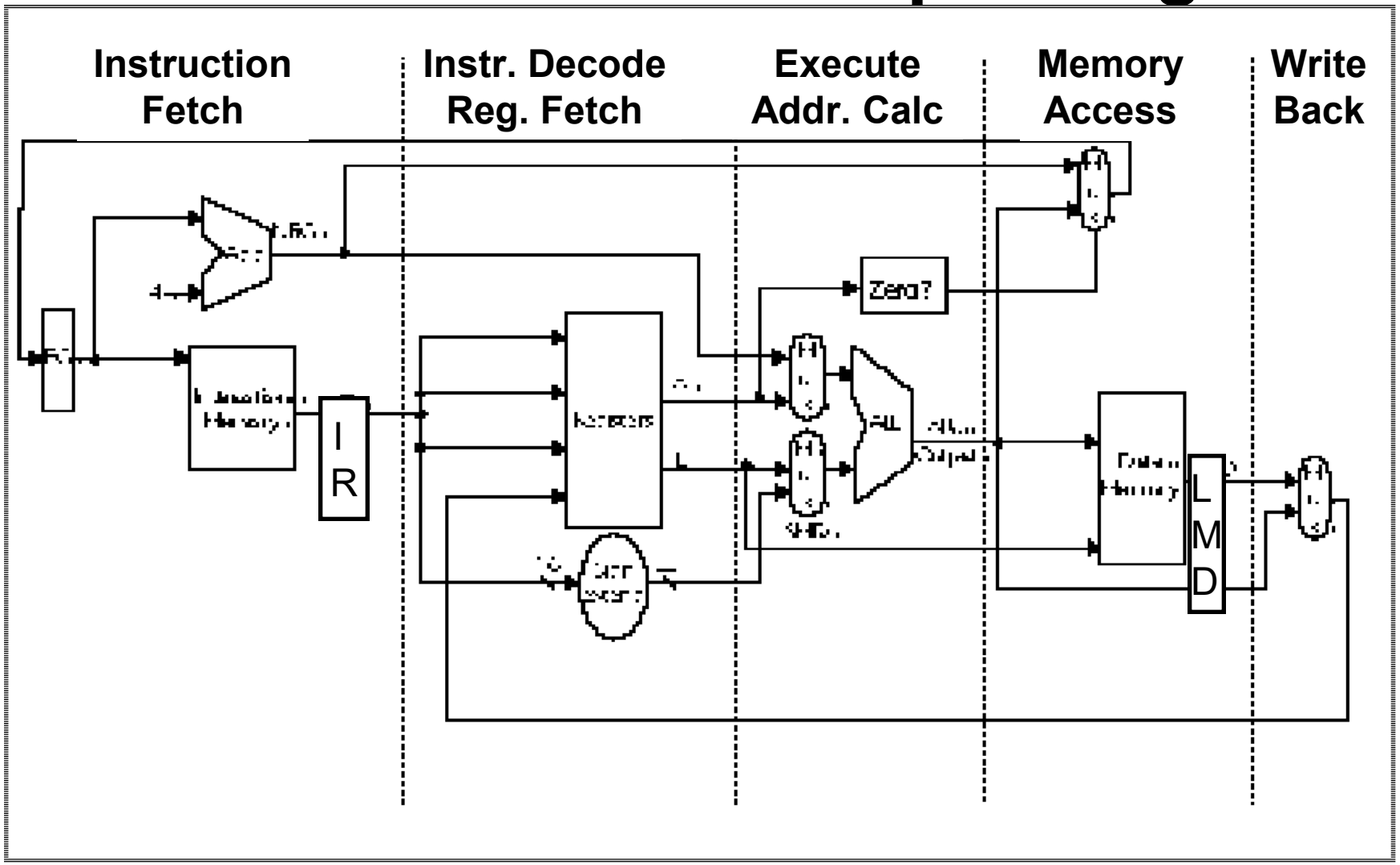


Pipelining Lessons

- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Pipeline rate limited by **slowest** pipeline stage
- **Multiple** tasks operating simultaneously
- Potential speedup = **Number pipe stages**
- Unbalanced lengths of pipe stages reduces speedup
- Time to "**fill**" pipeline and time to "**drain**" it reduces speedup

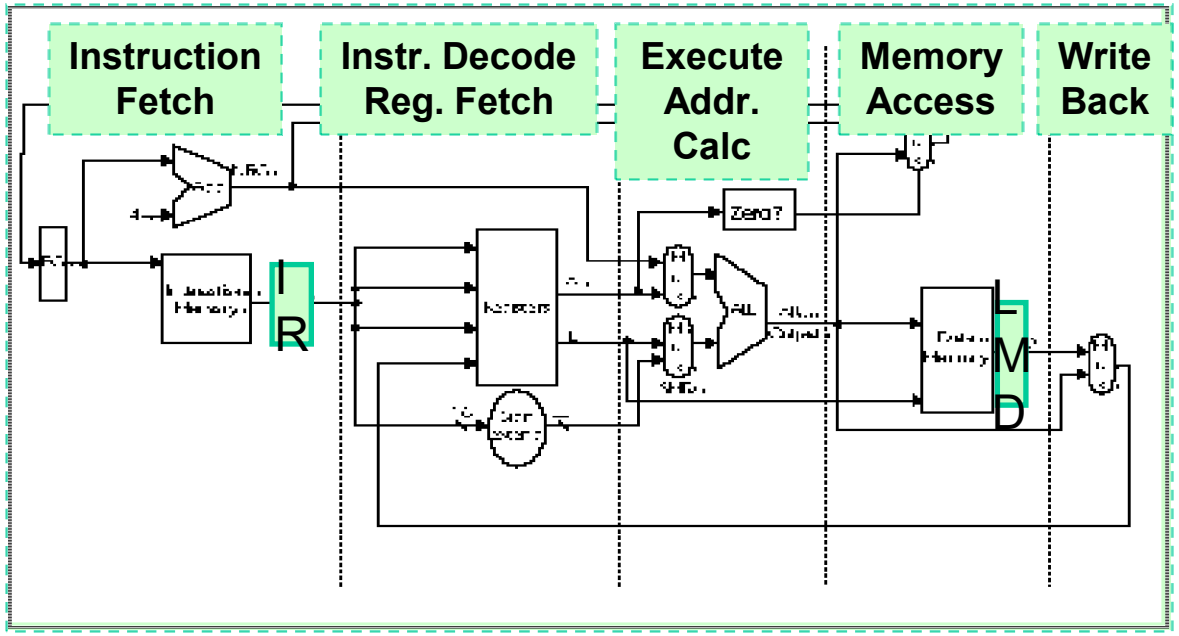
What Is Pipelining

MIPS Without Pipelining



What Is Pipelining

MIPS Functions



Passed To Next Stage

IR <- Mem[PC]

NPC <- PC + 4

Instruction Fetch (IF):

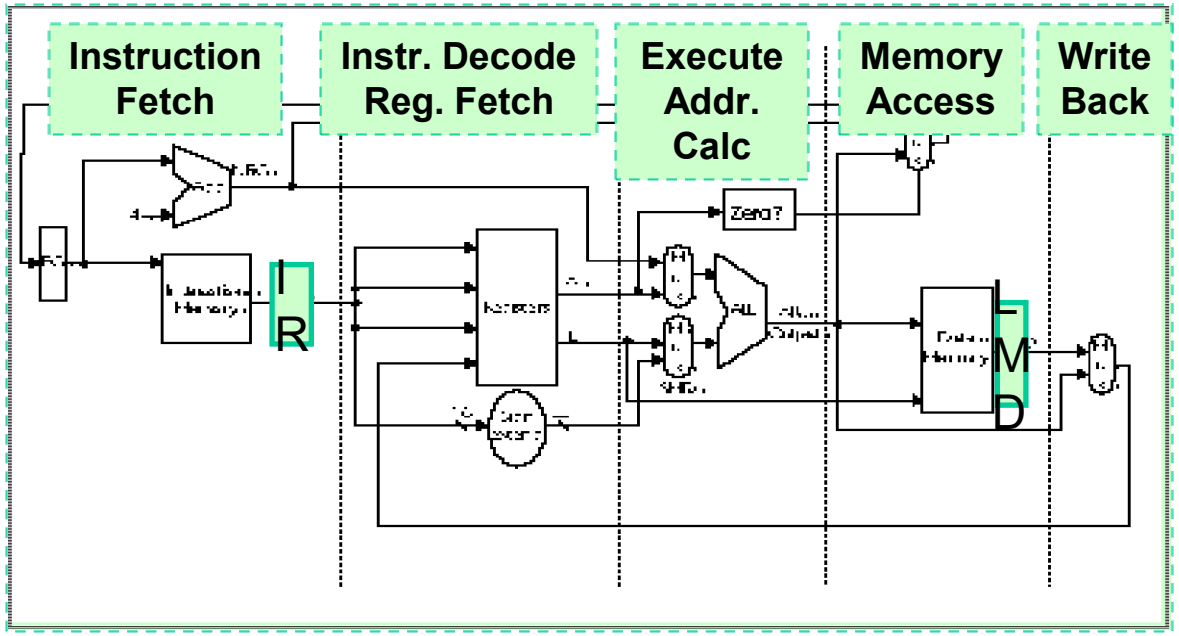
Send out the PC and fetch the instruction from memory into the instruction register (IR); increment the PC by 4 to address the next sequential instruction.

IR holds the instruction that will be used in the next stage.

NPC holds the value of the next PC.

What Is Pipelining

MIPS Functions



Passed To Next Stage

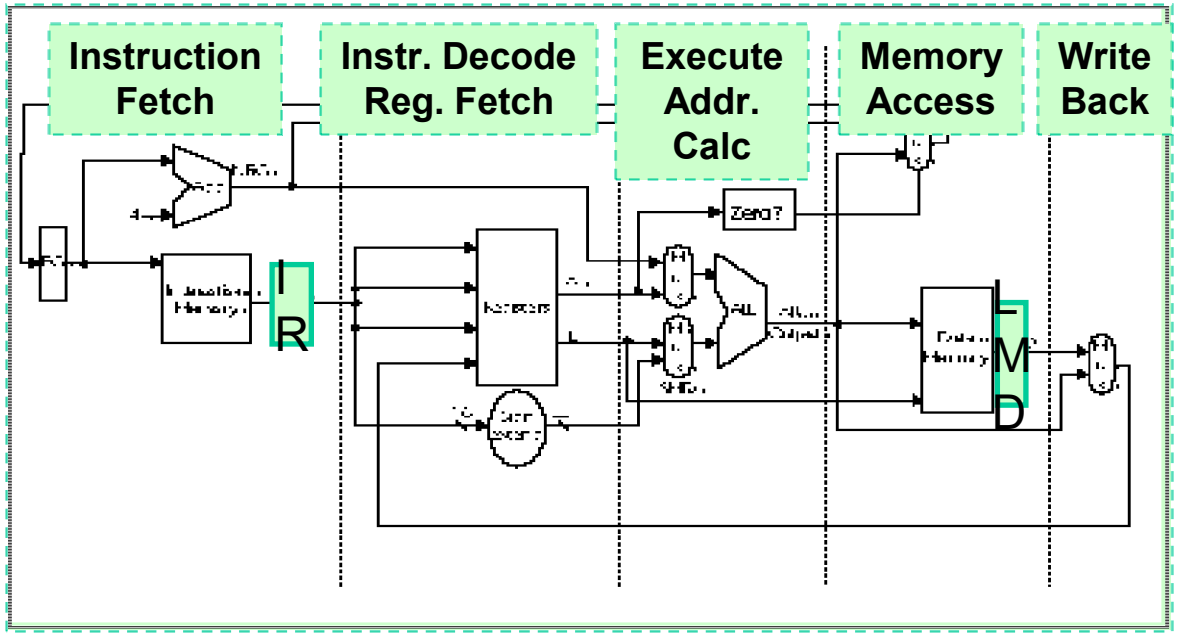
A <- Regs[IR6..IR10];
B <- Regs[IR10..IR15];
Imm <- ((IR16) ##IR16-31

Instruction Decode/Register Fetch Cycle (ID):

Decode the instruction and access the register file to read the registers.
The outputs of the general purpose registers are read into two temporary registers (A & B) for use in later clock cycles.
We extend the sign of the lower 16 bits of the Instruction Register.

What Is Pipelining

MIPS Functions



Passed To Next Stage

A <- A func. B

cond = 0;

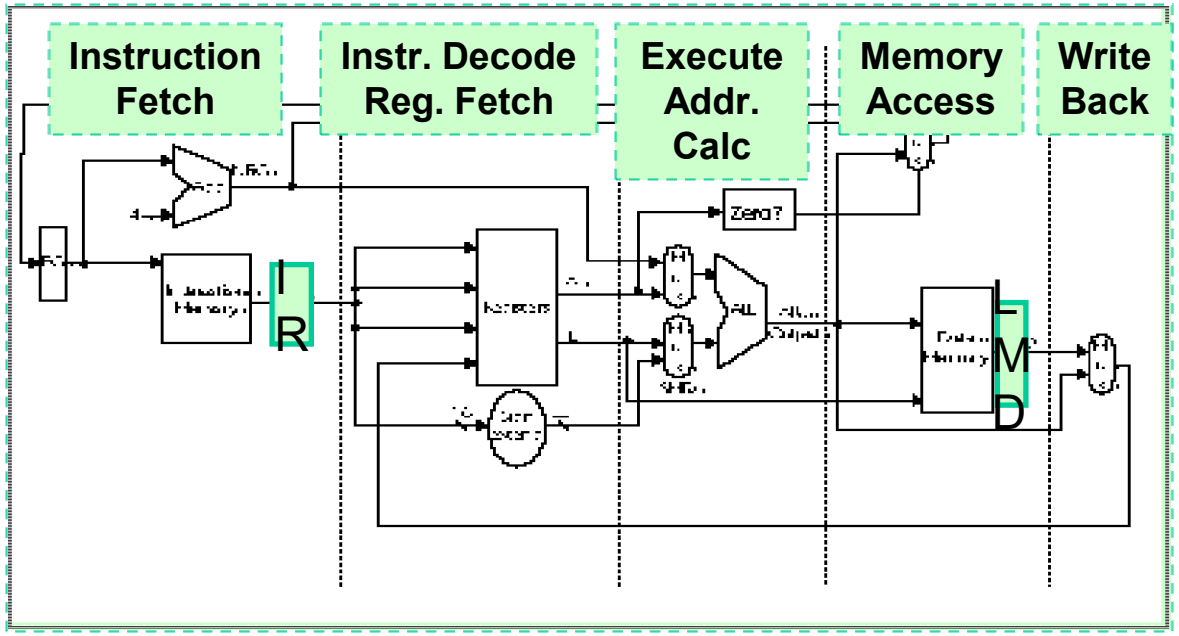
Execute Address Calculation (EX):

We perform an operation (for an ALU) or an address calculation (if it's a load or a Branch).

If an ALU, actually do the operation. If an address calculation, figure out how to obtain the address and stash away the location of that address for the next cycle.

What Is Pipelining

MIPS Functions



Passed To Next Stage

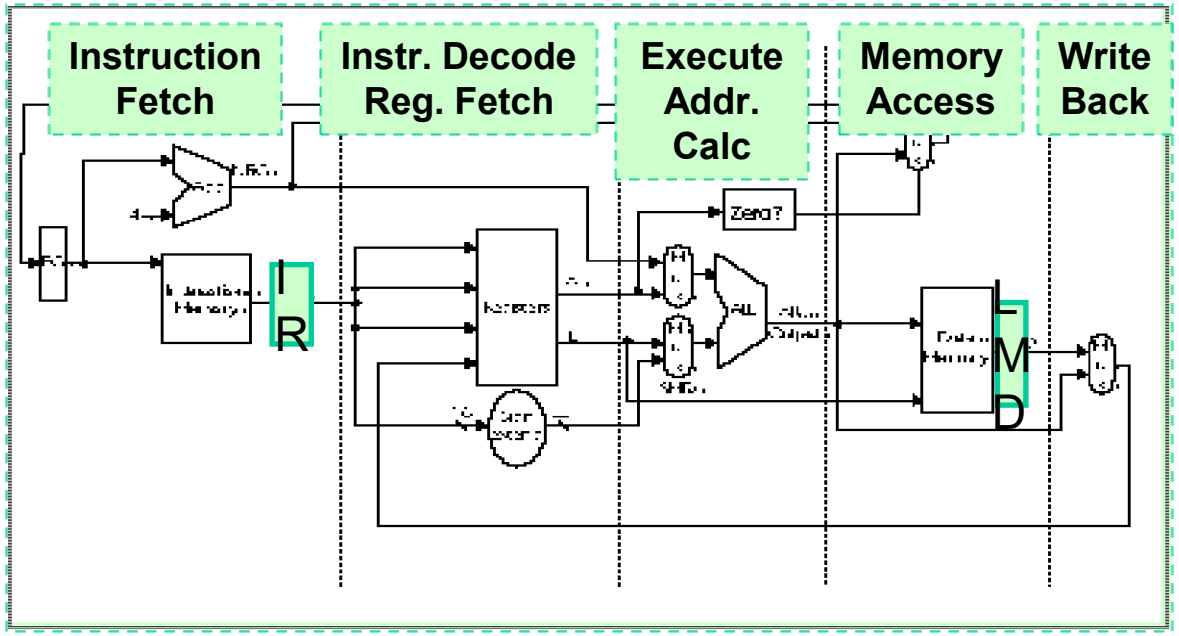
A = Mem[prev. B]
or
Mem[prev. B] = A

MEMORY ACCESS (MEM):

If this is an ALU, do nothing.
If a load or store, then access memory.

What Is Pipelining

MIPS Functions



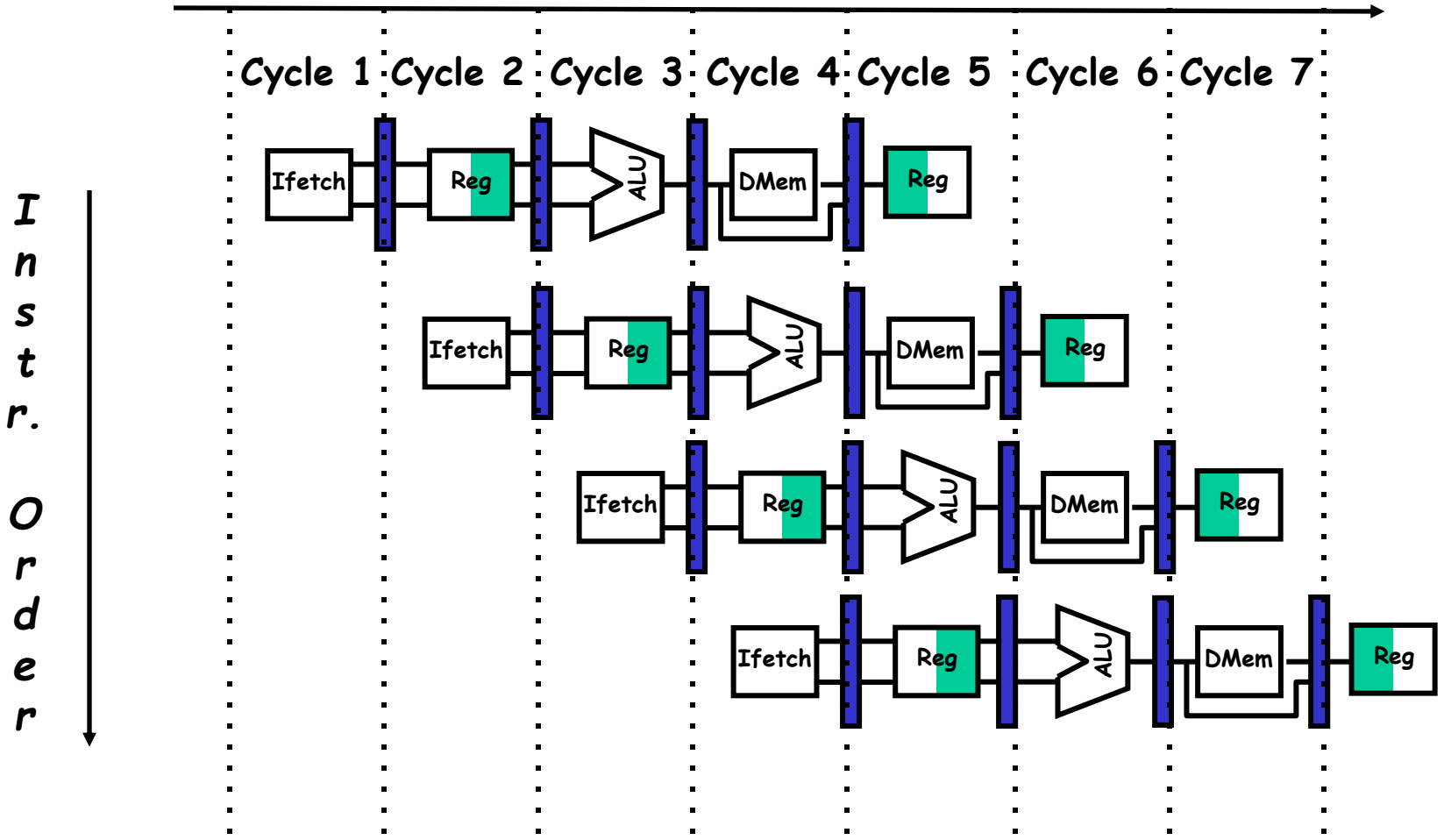
Passed To Next Stage

Regs <- A, B;

WRITE BACK (WB):

Update the registers from either the ALU or from the data loaded.

The Basic Pipeline For MIPS



Pipeline Hurdles

A.1 What is Pipelining?

A.2 The Major Hurdle of Pipelining-Structural Hazards

-- Structural Hazards

– Data Hazards

– Control Hazards

A.3 How is Pipelining Implemented

A.4 What Makes Pipelining Hard to Implement?

A.5 Extending the MIPS Pipeline to Handle Multi-cycle Operations

Limits to pipelining: Hazards prevent next instruction from executing during its designated clock cycle

- **Structural hazards:** HW cannot support this combination of instructions (single person to fold and put clothes away)
- **Data hazards:** Instruction depends on result of prior instruction still in the pipeline (missing sock)
- **Control hazards:** Pipelining of branches & other instructions that change the PC
- Common solution is to **stall** the pipeline until the hazard is resolved, inserting one or more "**bubbles**" in the pipeline

Pipeline Hurdles

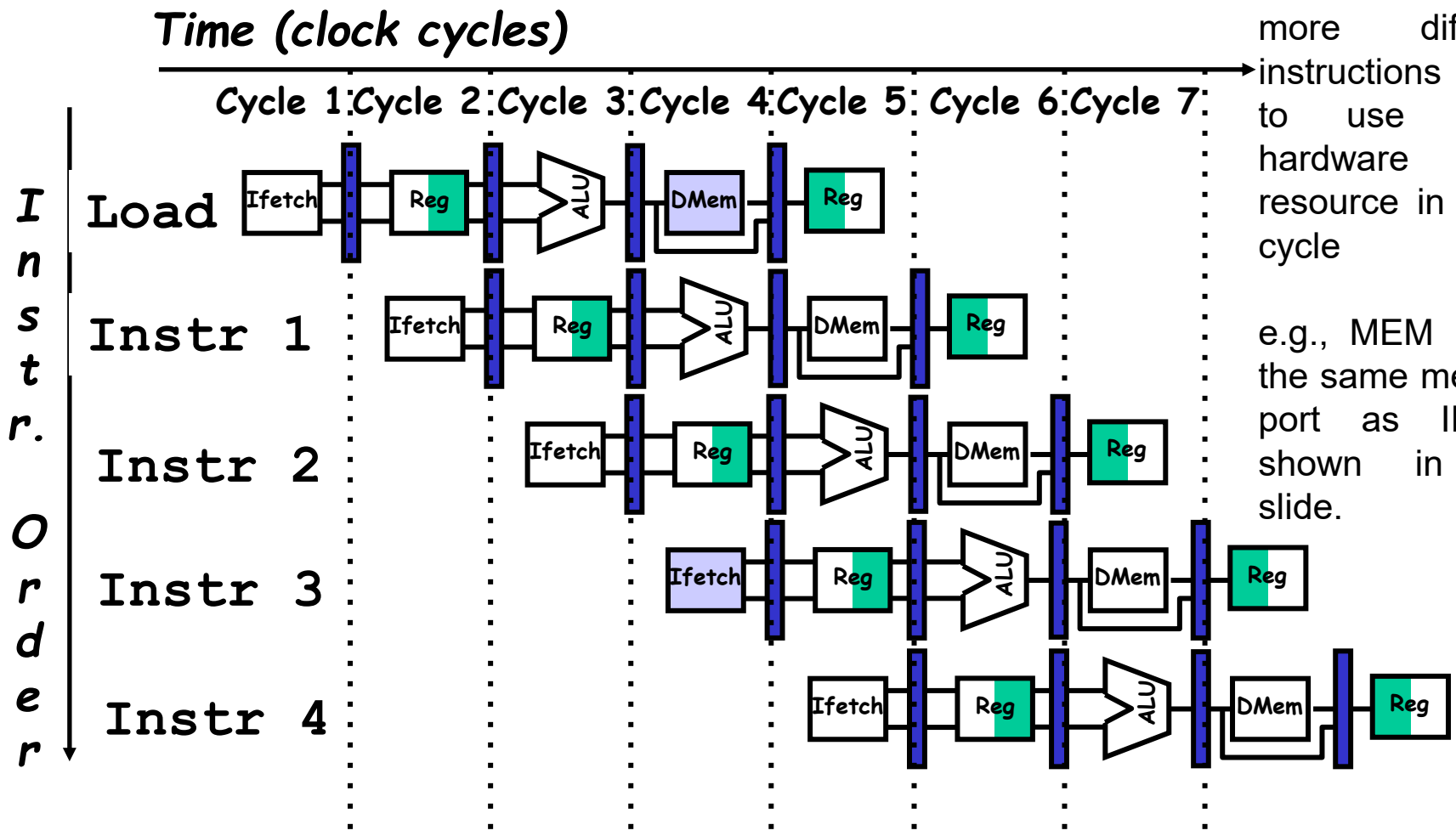
Definition

- conditions that lead to incorrect behavior if not fixed
- Structural hazard
  - two different instructions use same h/w in same cycle
- Data hazard
  - two different instructions use same storage
  - must appear as if the instructions execute in correct order
- Control hazard
  - one instruction affects which instruction is next

Resolution

- Pipeline interlock logic detects hazards and fixes them
- simple solution: stall
- increases CPI, decreases performance
- better solution: partial stall
- some instruction stall, others proceed better to stall early than late

Structural Hazards



When two or more different instructions want to use same hardware resource in same cycle

e.g., MEM uses the same memory port as IF as shown in this slide.

Figure 3.6

Structural Hazards

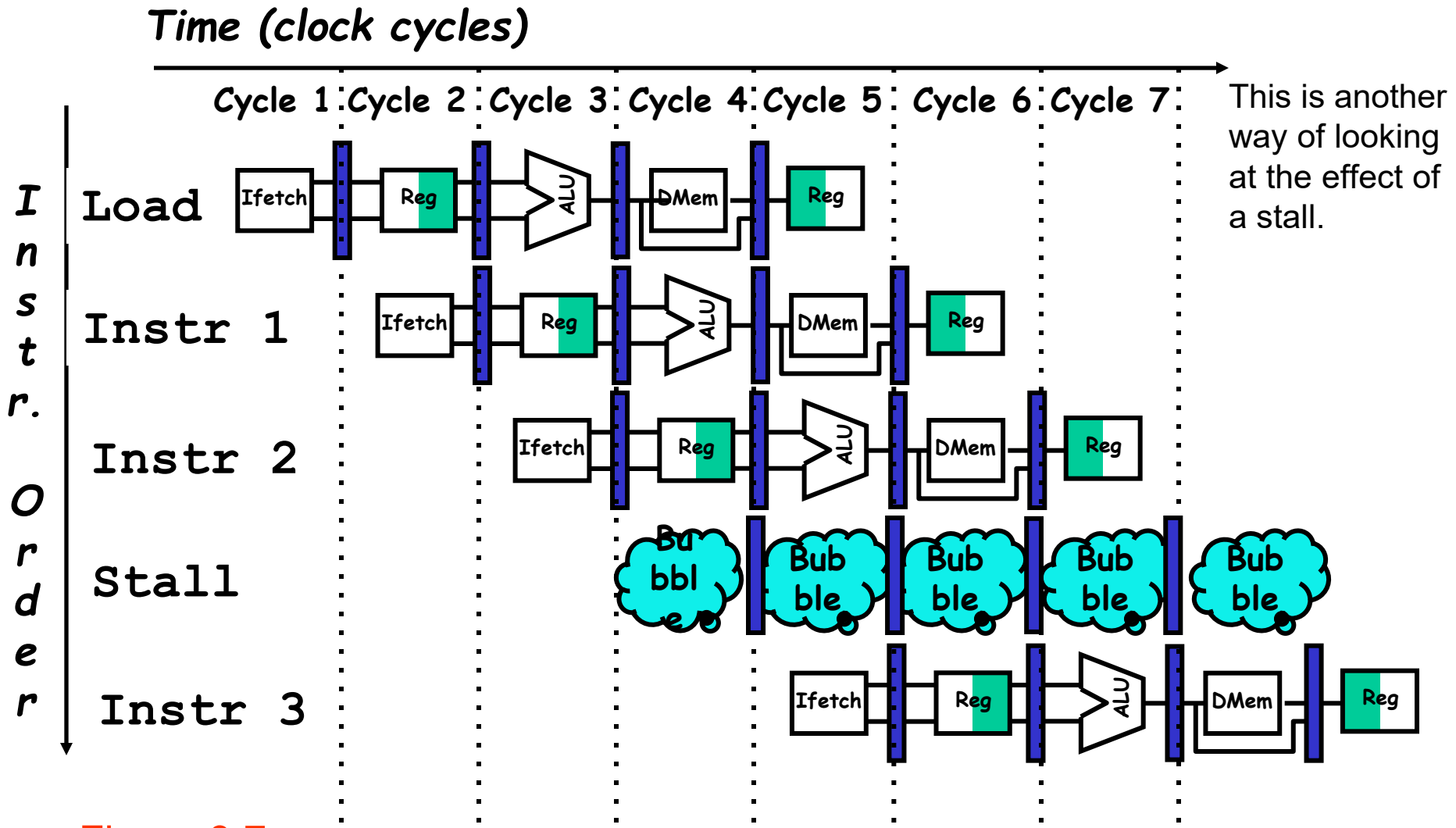


Figure 3.7

Structural Hazards

	Clock cycle number									
Instruction	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction i + 1		IF	ID	EX	MEM	WB				
Instruction i + 2			IF	ID	EX	MEM	WB			
Instruction i + 3				stall	IF	ID	EX	MEM	WB	
Instruction i + 4						IF	ID	EX	MEM	WB
Instruction i + 5							IF	ID	EX	MEM
Instruction i + 6								IF	ID	EX

This is another way to represent the stall we saw on the last few pages.

Structural Hazards

Dealing with Structural Hazards

Stall

- low cost, simple
- Increases CPI
- use for rare case since stalling has performance effect

Pipeline hardware resource

- useful for multi-cycle resources
- good performance
- sometimes complex e.g., RAM

Replicate resource

- good performance
- increases cost (+ maybe interconnect delay)
- useful for cheap or divisible resources

Structural Hazards

Structural hazards are reduced with these rules:

- Each instruction uses a resource at most once
- Always use the resource in the same pipeline stage
- Use the resource for one cycle only

Many RISC ISA'a designed with this in mind

Sometimes very complex to do this. For example, memory of necessity is used in the IF and MEM stages.

Some common Structural Hazards:

- Memory - we've already mentioned this one.
- Floating point - Since many floating point instructions require many cycles, it's easy for them to interfere with each other.
- Starting up more of one type of instruction than there are resources. For instance, the PA-8600 can support two ALU + two load/store instructions per cycle - that's how much hardware it has available.

Structural Hazards

This is the example on Page 144.

We want to compare the performance of two machines. Which machine is faster?

- Machine A: Dual ported memory - so there are no memory stalls
- Machine B: Single ported memory, but its pipelined implementation has a 1.05 times faster clock rate

Assume:

- Ideal CPI = 1 for both
- Loads are 40% of instructions executed

$$\text{SpeedUp}_A = \text{Pipeline Depth}/(1 + 0) \times (\text{clock}_{\text{unpipe}}/\text{clock}_{\text{pipe}})$$
$$= \text{Pipeline Depth}$$
$$\text{SpeedUp}_B = \text{Pipeline Depth}/(1 + 0.4 \times 1) \times (\text{clock}_{\text{unpipe}}/(\text{clock}_{\text{unpipe}} / 1.05)$$
$$= (\text{Pipeline Depth}/1.4) \times 1.05$$
$$= 0.75 \times \text{Pipeline Depth}$$

SpeedUp_A / SpeedUp_B = Pipeline Depth / (0.75 x Pipeline Depth) = 1.33

- Machine A is 1.33 times faster

Data Hazards

A.1 What is Pipelining?

A.2 The Major Hurdle of Pipelining-Structural Hazards

-- Structural Hazards

– Data Hazards

– Control Hazards

A.3 How is Pipelining Implemented

A.4 What Makes Pipelining Hard to Implement?

A.5 Extending the MIPS Pipeline to Handle Multi-cycle Operations

These occur when at any time, there are instructions active that need to access the same data (memory or register) locations.

Where there's real trouble is when we have:

instruction A
instruction B

and B manipulates (reads or writes) data before A does. This violates the order of the instructions, since the architecture implies that A completes entirely before B is executed.

Data Hazards

Execution Order is:
$Instr_I$
$Instr_J$

Read After Write (RAW)

$Instr_J$ tries to read operand before $Instr_I$ writes it



```
I: add r1,r2,r3
J: sub r4,r1,r3
```

- **Caused by a "Dependence" (in compiler nomenclature). This hazard results from an actual need for communication.**

Data Hazards

Execution Order is:
$Instr_I$
$Instr_J$

Write After Read (WAR)

$Instr_J$ tries to write operand *before* $Instr_I$ reads i

- Gets wrong operand



```
I: sub r4,r1,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

- Called an "anti-dependence" by compiler writers. This results from reuse of the name "r1".

- **Can't happen in MIPS 5 stage pipeline because:**
  - All instructions take 5 stages, and
  - Reads are always in stage 2, and
  - Writes are always in stage 5

Data Hazards

Execution Order is:
$Instr_I$
$Instr_J$

Write After Write (WAW)

$Instr_J$ tries to write operand *before* $Instr_I$ writes it

- Leaves wrong result ($Instr_I$ not $Instr_J$)



```
I: sub r1,r4,r3
J: add r1,r2,r3
K: mul r6,r1,r7
```

- **Called an "output dependence" by compiler writers**
  This also results from the reuse of name "r1".
- **Can't happen in MIPS 5 stage pipeline because:**
  - All instructions take 5 stages, and
  - Writes are always in stage 5
- **Will see WAR and WAW in later more complicated pipes**

Data Hazards

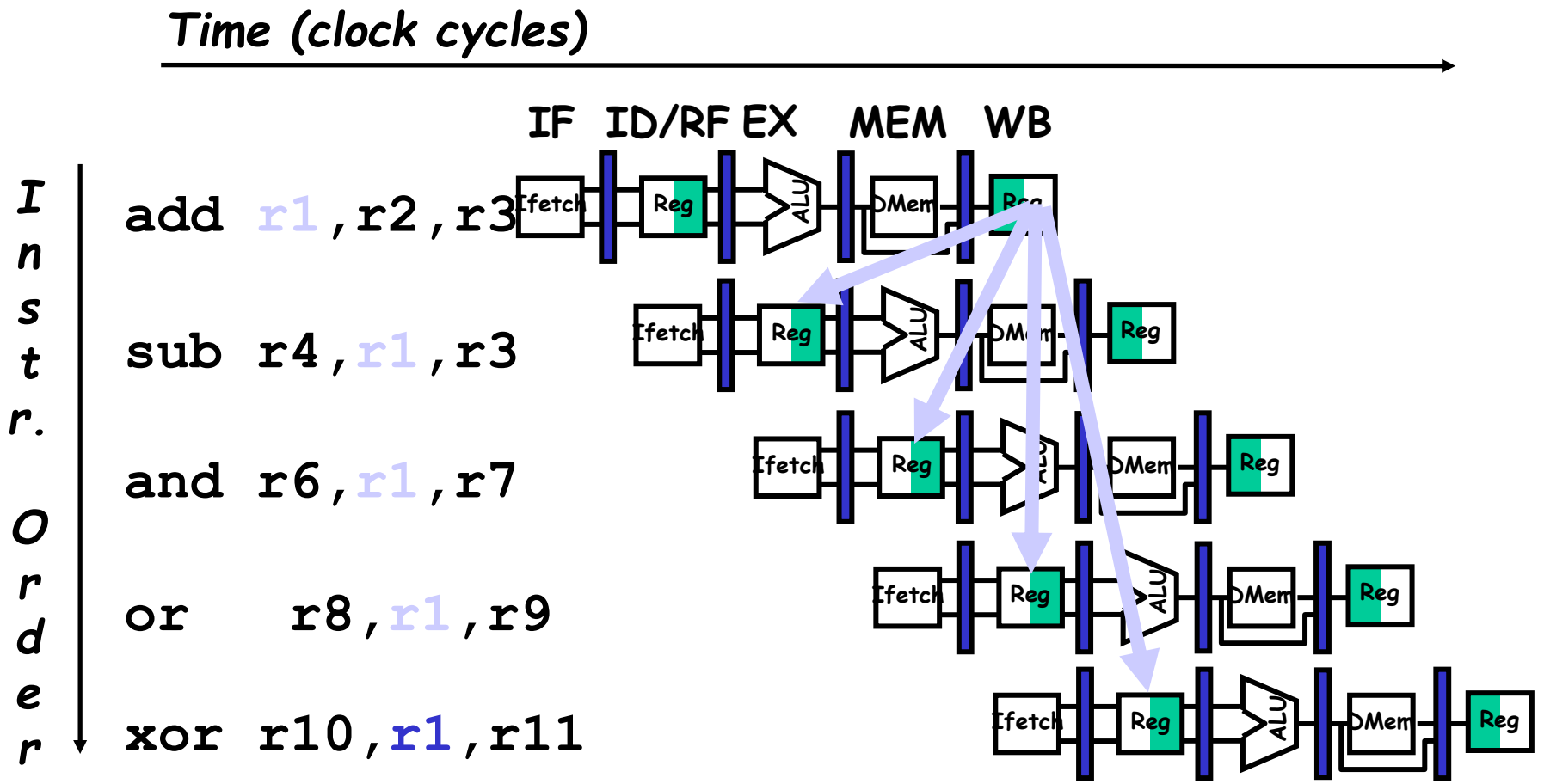
Simple Solution to RAW

- Hardware detects RAW and stalls
- Assumes register written then read each cycle
  + low cost to implement, simple
  -- reduces IPC
- Try to minimize stalls

Minimizing RAW stalls

- Bypass/forward/shortcircuit (We will use the word “forward”)
- Use data before it is in the register
  + reduces/avoids stalls
  -- complex
- Crucial for common RAW hazards

Data Hazards



The use of the result of the ADD instruction in the next three instructions causes a hazard, since the register is not written until after those instructions read it.

Figure 3.9

Data Hazards

Forwarding To Avoid Data Hazard

Forwarding is the concept of making data available to the input of the ALU for subsequent instructions, even though the generating instruction hasn't gotten to WB in order to write the memory or registers.

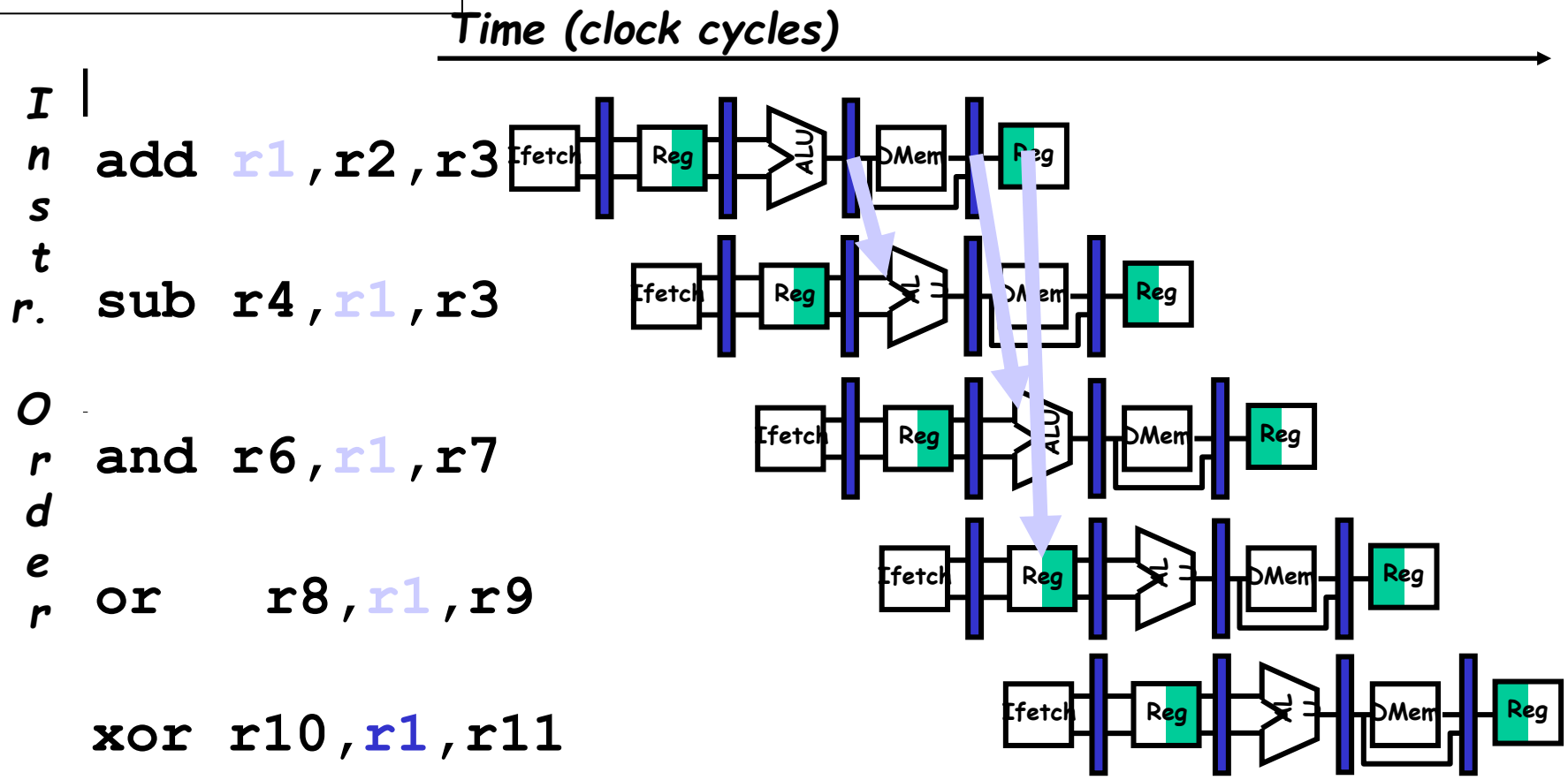
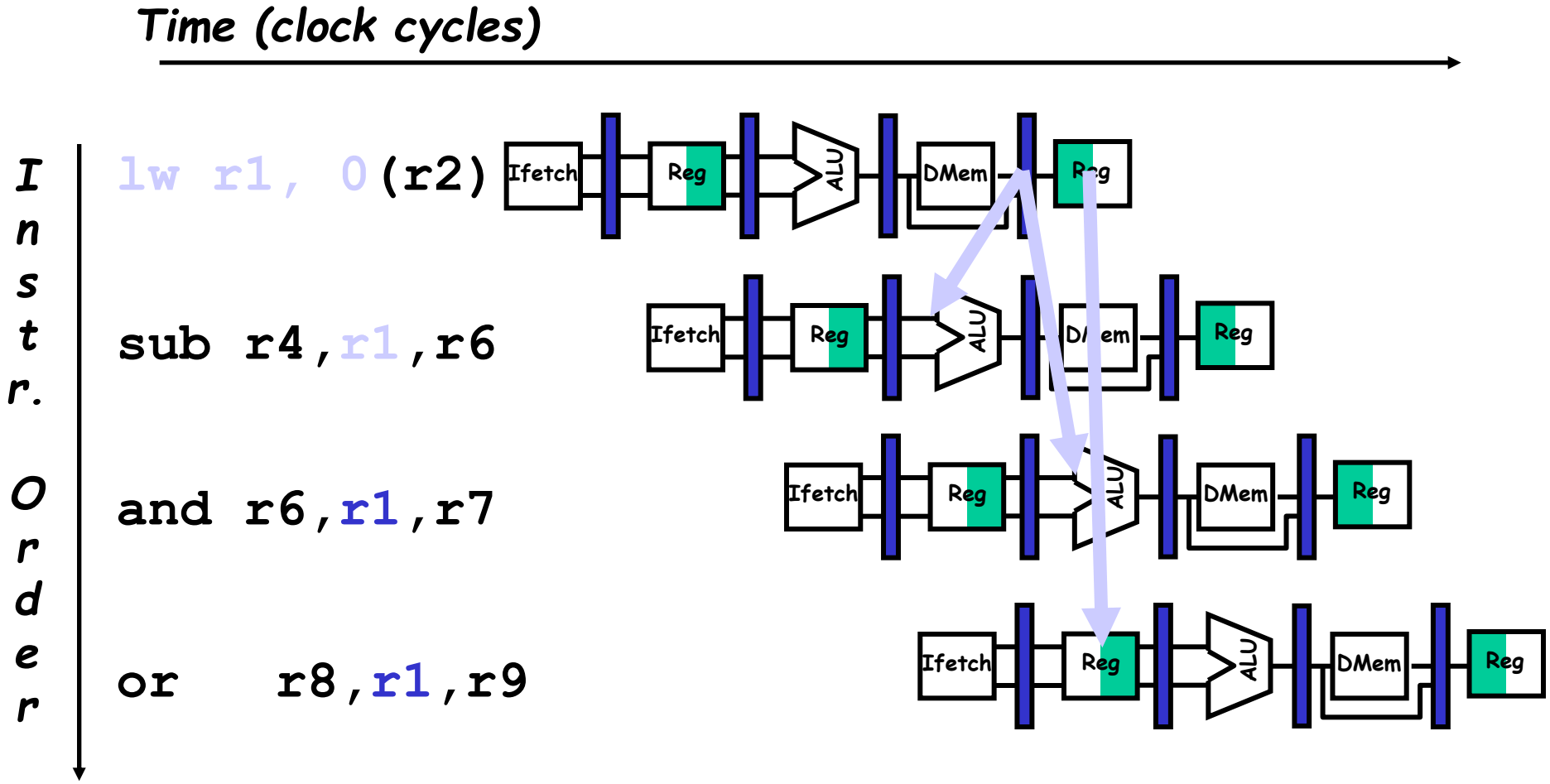


Figure 3.10

Data Hazards

The data isn't loaded until after the MEM stage.

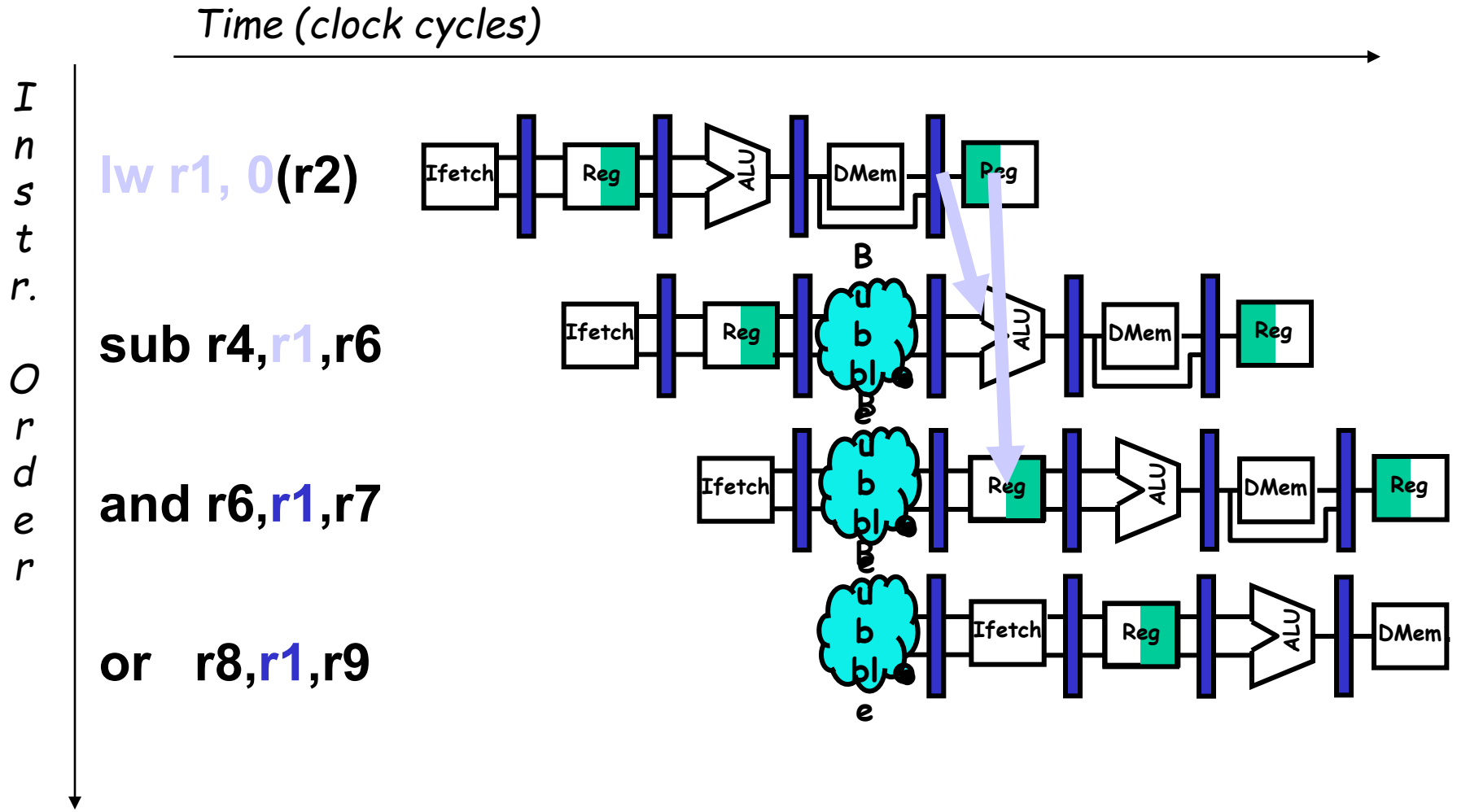


There are some instances where hazards occur, even with forwarding.

Figure 3.12

Data Hazards

The stall is necessary as shown here.



There are some instances where hazards occur, even with forwarding.

Figure 3.13

Data Hazards

This is another representation of the stall.

LW R1, 0(R2)	IF	ID	EX	MEM	WB			
SUB R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
OR R8, R1, R9				IF	ID	EX	MEM	WB

LW R1, 0(R2)	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR R8, R1, R9				stall	IF	ID	EX	MEM	WB

Data Hazards

Instruction scheduled by compiler - move instruction in order to reduce stall.

```
lw Rb, b                code sequence for a = b+c before scheduling
lw Rc, c
Add Ra, Rb, Rc          stall
sw a, Ra
lw Re, e                code sequence for d = e+f before scheduling
lw Rf, f
sub Rd, Re, Rf          stall
sw d, Rd
```

Arrangement of code after scheduling.

```
lw Rb, b
lw Rc, c
lw Re, e
Add Ra, Rb, Rc
lw Rf, f
sw a, Ra
sub Rd, Re, Rf
sw d, Rd
```

A.1 What is Pipelining?

A.2 The Major Hurdle of Pipelining-Structural Hazards

-- Structural Hazards

– Data Hazards

– Control Hazards

A.3 How is Pipelining Implemented

A.4 What Makes Pipelining Hard to Implement?

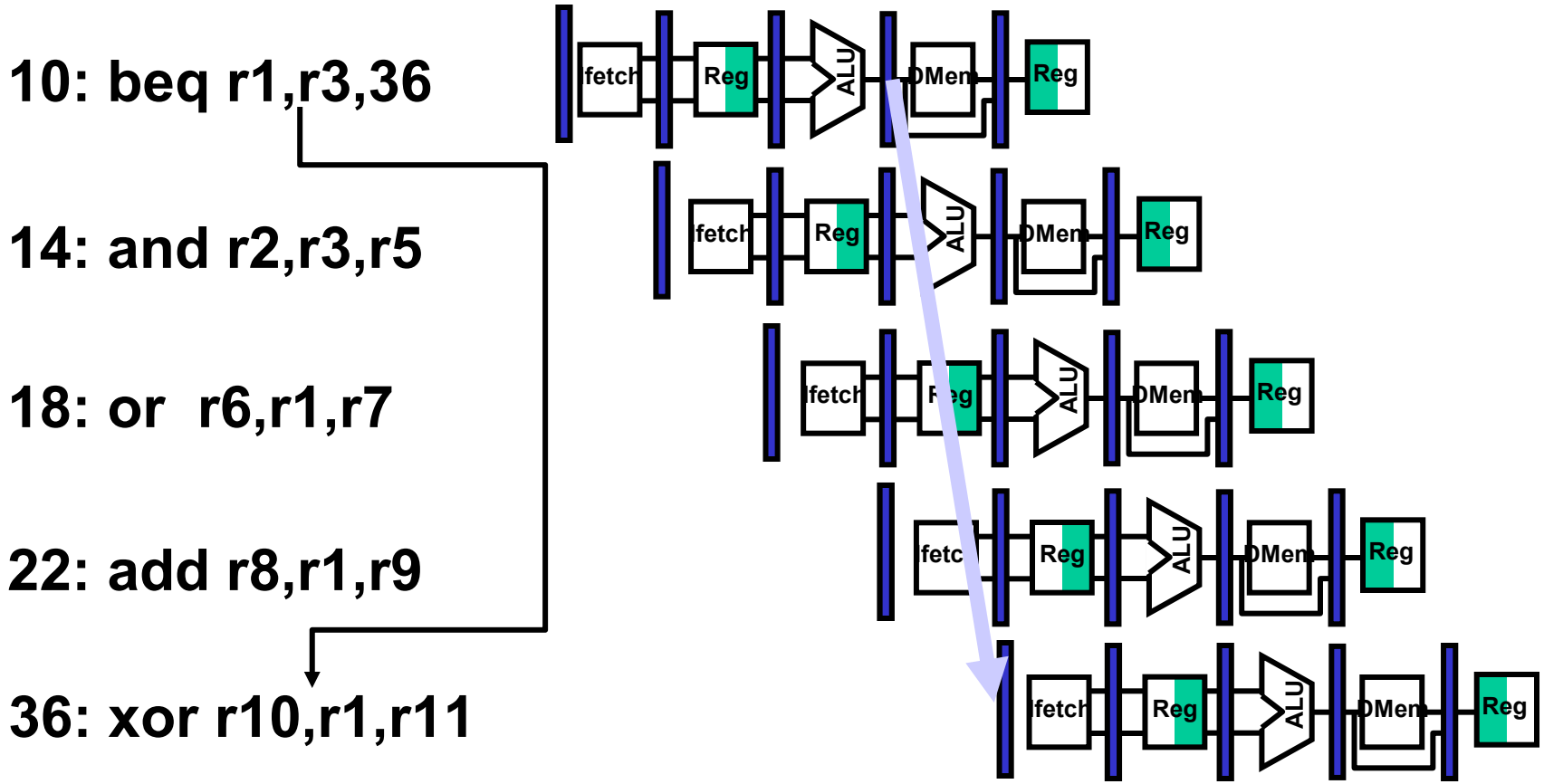
A.5 Extending the MIPS Pipeline to Handle Multi-cycle Operations

Control Hazards

A control hazard is when we need to find the destination of a branch, and can't fetch any new instructions until we know that destination.

Control Hazards

Control Hazard on Branches Three Stage Stall



- Branch hazards are caused by branch instructions and are known as control hazards.
- The flow of program/instruction execution is controlled by branch instructions.
- In higher-level languages, conditional statements are used for repetitive loops or condition testing (correlate with while, for, if, case statements).
- These are converted into one of the BRANCH instruction variations.
- To understand the programme flow, you must know the value of the condition being tested. It's a difficult situation.
- As a result, when the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch, which examines the condition's consequent value, a conditional hazard develops.
- The Program Counter (PC) is loaded with the appropriate place for the branch and jump instructions, which determines the programme flow. The next instruction that is to be fetched and executed by the given CPU is stored in the PC.

Conditional Hazards Solutions

1. Stall

Stall the given pipeline as soon as any branch instructions are decoded. Stalling reduces throughput as it always does. According to statistics, at least 30% of the instructions in a program are BRANCH. With Stalling, the pipeline is effectively operating at 50% capacity.

2. Prediction

Consider a for or a while loop that is repeated 100 times. We know the programme would run 100 times without the given branch condition being met. The program only exits the loop for the 101st time. As a result, it’s better to let the pipeline run its course and then flush/undo when the branch condition is met. This has less of an impact on the pipeline’s throttle and stalling.

3. Dynamic Branch Prediction

With the help of Branch Table Buffer, a historical record is kept (BTB). The BTB is a type of cache that contains a series of entries containing the branch instruction's PC address and the effective branch address. This is done for each branch instruction that is encountered. When a conditional branch instruction is encountered, the BTB is queried for the matching branch instruction address. If the target branch address is hit, the next instruction is fetched from the associated target branch address. Dynamic branch prediction is the term for this.

Branch Instruction Address	Target Branch Address Taken

4. Reordering Instructions

Delayed branching entails reordering the instructions to move the branch instruction later in the sequence, allowing safe and beneficial instructions that are unaffected by the result of a branch to be brought in earlier in the sequence, delaying the fetch of the branch instruction. If such instructions are not available, NOP is used. The Compiler is used to implement this delayed branch.

Memory Organization & Types

- There are two kinds of main memory: *random access memory, RAM, and read-only-memory, ROM*.
- There are two types of RAM, dynamic RAM (DRAM) and static RAM (SRAM).
- DRAM consists of capacitors that slowly leak their charge over time. Thus, they must be refreshed every few milliseconds to prevent data loss.
- DRAM is “cheap” memory owing to its simple design.

Memory Organization & Types

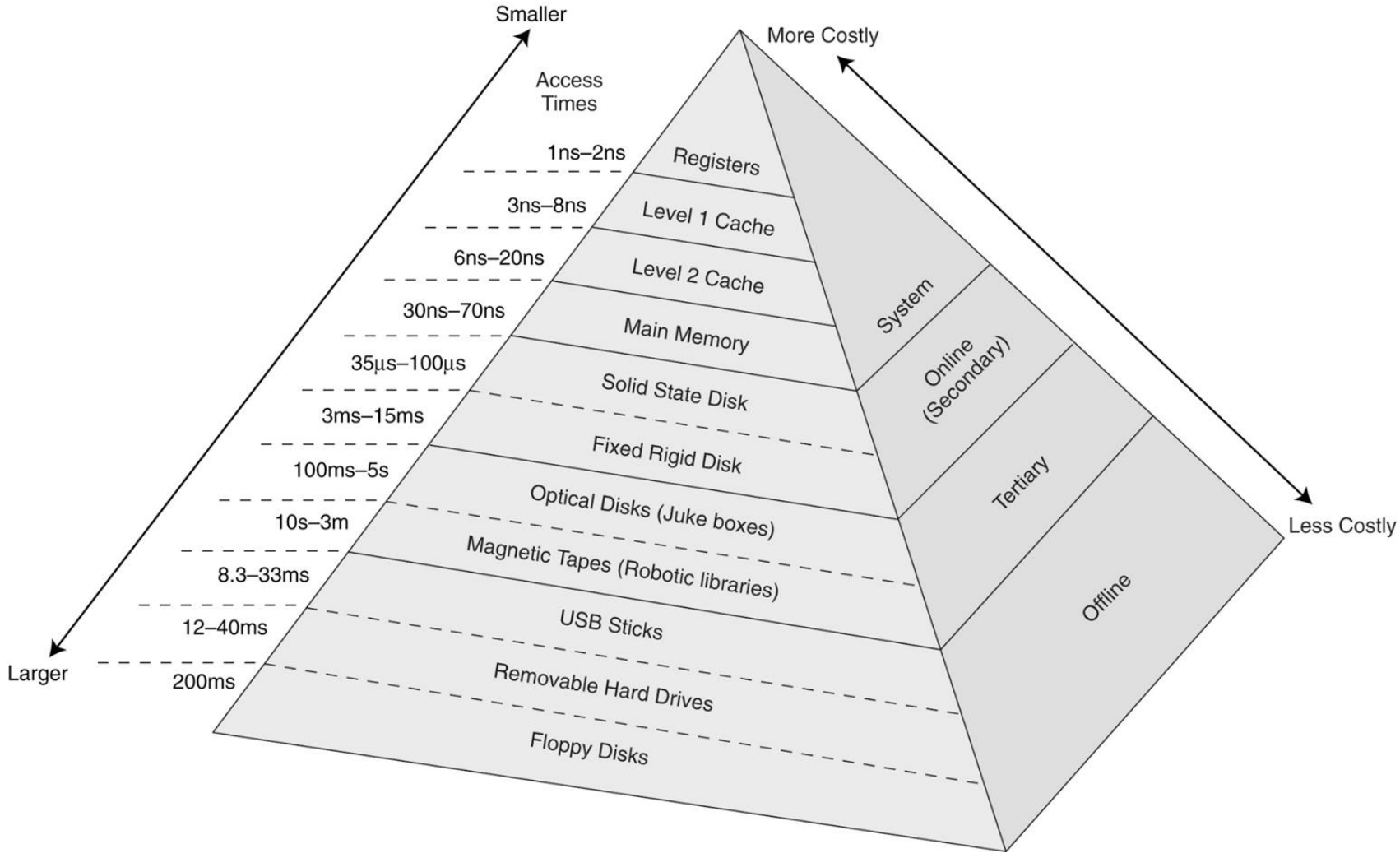
- SRAM consists of circuits similar to the D flip-flop.
- SRAM is very fast memory and it doesn’t need to be refreshed like DRAM does. It is used to build cache memory, which we will discuss in detail later.
- ROM also does not need to be refreshed, either. In fact, it needs very little charge to retain its memory.
- ROM is used to store permanent, or semi-permanent data that persists even while the system is turned off.

The Memory Hierarchy

- Generally speaking, faster memory is more expensive than slower memory.
- To provide the best performance at the lowest cost, memory is organized in a hierarchical fashion.
- Small, fast storage elements are kept in the CPU, larger, slower main memory is accessed through the data bus.
- Larger, (almost) permanent storage in the form of disk and tape drives is still further from the CPU.

The Memory Hierarchy

- This storage organization can be thought of as a pyramid:



The Memory Hierarchy

- We are most interested in the memory hierarchy that involves registers, cache, main memory, and virtual memory.
- Registers are storage locations available on the processor itself.
- Virtual memory is typically implemented using a hard drive; it extends the address space from RAM to the hard drive.
- Virtual memory provides more space: Cache memory provides speed.

The Memory Hierarchy

- To access a particular piece of data, the CPU first sends a request to its nearest memory, usually cache.
- If the data is not in cache, then main memory is queried. If the data is not in main memory, then the request goes to disk.
- Once the data is located, then the data, and a number of its nearby data elements are fetched into cache memory.

The Memory Hierarchy

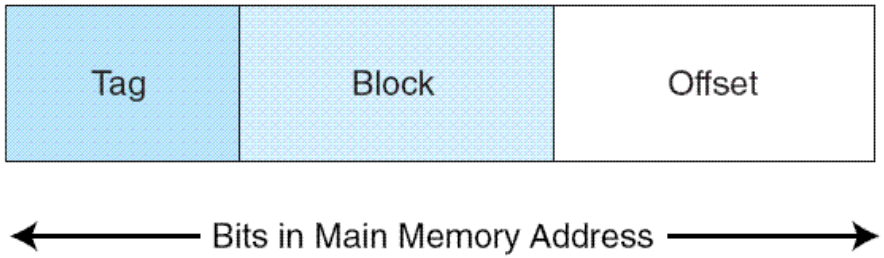
- This leads us to some definitions.
  - A *hit* is when data is found at a given memory level.
  - A *miss* is when it is not found.
  - The *hit rate* is the percentage of time data is found at a given memory level.
  - The *miss rate* is the percentage of time it is not.
  - Miss rate = 1 - hit rate.
  - The *hit time* is the time required to access data at a given memory level.
  - The *miss penalty* is the time required to process a miss, including the time that it takes to replace a block of memory plus the time it takes to deliver the data to the processor.

Cache Memory

- The purpose of cache memory is to speed up accesses by storing recently used data closer to the CPU, instead of storing it in main memory.
- Although cache is much smaller than main memory, its access time is a fraction of that of main memory.
- Unlike main memory, which is accessed by address, cache is typically accessed by content; hence, it is often called *content addressable memory*.
- Because of this, a single large cache memory isn't always desirable-- it takes longer to search.

Cache Memory

- The “content” that is addressed in content addressable cache memory is a subset of the bits of a main memory address called a *field*.
  - Many blocks of main memory map to a single block of cache. A *tag* field in the cache block distinguishes one cached memory block from another.
  - A *valid bit* indicates whether the cache block is being used.
  - An *offset field* points to the desired data in the block.



Cache Memory

- The simplest cache mapping scheme is *direct mapped cache*.
- In a direct mapped cache consisting of *N* blocks of cache, block *X* of main memory maps to cache block $Y = X \bmod N$.
- Thus, if we have 10 blocks of cache, block 7 of cache may hold blocks 7, 17, 27, 37, . . . of main memory.

- With direct mapped cache consisting of *N* blocks of cache, block *X* of main memory maps to cache block *Y* = *X* mod *N*.

